

LECTURE - 1

Tuesday
20/8/25.

- Set • Subset • Compliment • Intersection • Union
- Difference • Equal:

$$\overline{A \cup B} = \bar{A} \cap \bar{B} \quad (\text{Demorgan's law})$$

$$\overline{A \cap B} = \bar{A} \cup \bar{B} \quad (U - A) \cup (U - B)$$

$$A \cup A = A$$

$$\overline{\bar{A}} = A$$

$$A \cup \phi = A$$

$$A \cap \phi = \{ \} \quad (\text{empty set}) \rightarrow \text{has length} = 0$$

* $\{ \Lambda \}$: Null set whose length is zero but has a character ' Λ ' inside it to show NULL.

$$\Sigma = \{ a, b \}$$

Σ^* → Kleene star: all possible combinations

$$\Sigma^* = \{ \Lambda, a, b, aa, ab, bb, ba, aaa, \dots \}$$

length: 0 1 2 3

$$\Sigma^+ = \Sigma^* - \{ \Lambda \} = \{ a, b, aa, ab, \dots \}$$

⇒ Making new language:

$$Q: L_1 = \{ u \mid u \in \Sigma^* \text{ and } u \text{ starts with } a \}$$

$$L_1 = \{ a, ab, aa, aaa, aab, \dots \}$$

LECTURE - 2

day / date:
Thurs day
21/8/25.

Q: Enumerate the following:
 $L_1 = \{x \mid x \in \Sigma^* \text{ \& every } a \text{ in } x \text{ (if any) is followed by at least 2 b's}\}$

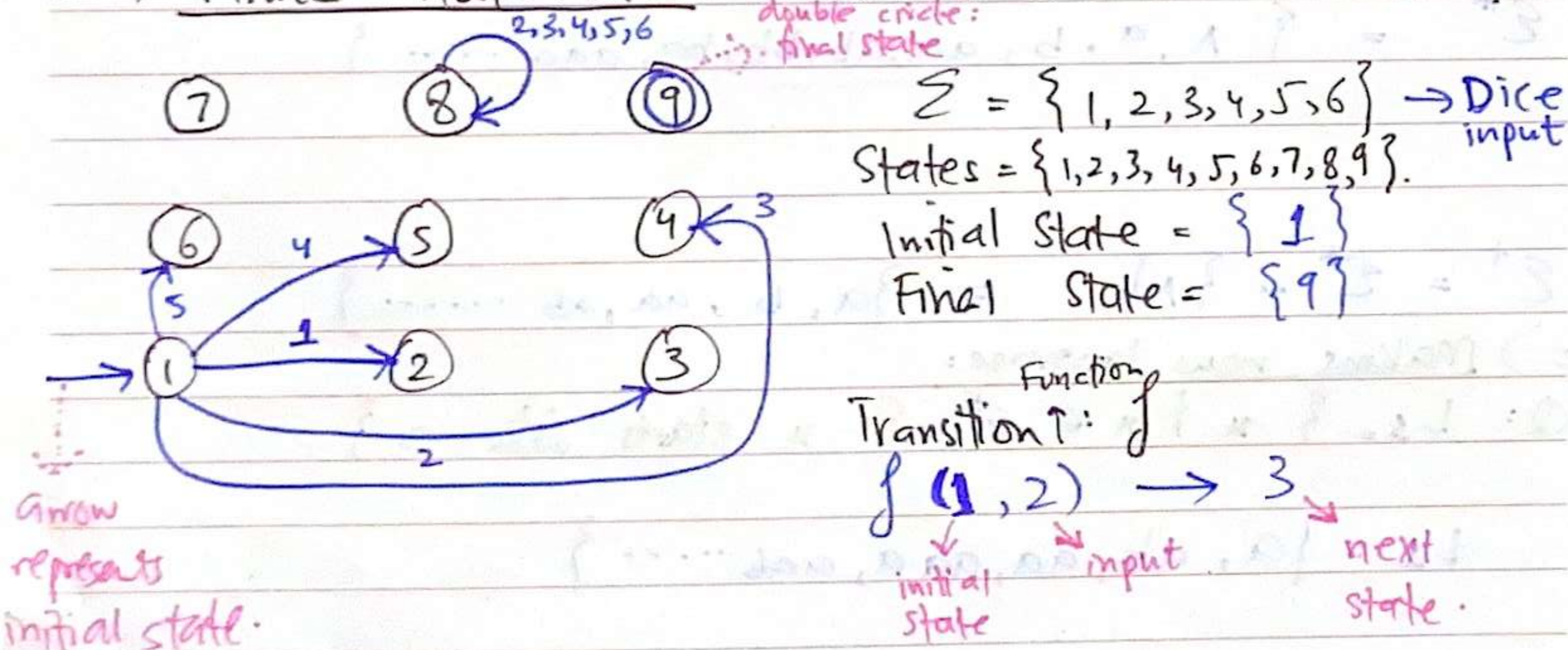
$L_1 = \{1, b, bb, bbb, \dots, labb, abbb, babb, abbabb, \dots\}$

$L_2 = \{x \mid x \in \Sigma^* \text{ \& } n_a(x) > n_b(x)\}$
 $n_a(x)$ = No. of a's in x
 $n_b(x)$ = No. of b's in x.

$L_2 = \{a, aab, aba, baa, aaa, \dots\}$

$L_3 = \{x \mid x \in \Sigma^* \text{ \& } x = a^*\} \rightarrow \{1, a, aa, aaa, \dots\}$
 $\{1, a, aa, aaa, aaaa, \dots\}$

$\Rightarrow$ Finite Automata: "Snakes & Ladder Board"



$\Rightarrow$ DFA

we know final values in memory (it's not like n=0)
DETERMINISTIC

FINITE AUTOMATA: $(Q, \Sigma, q_0, A, \delta)$

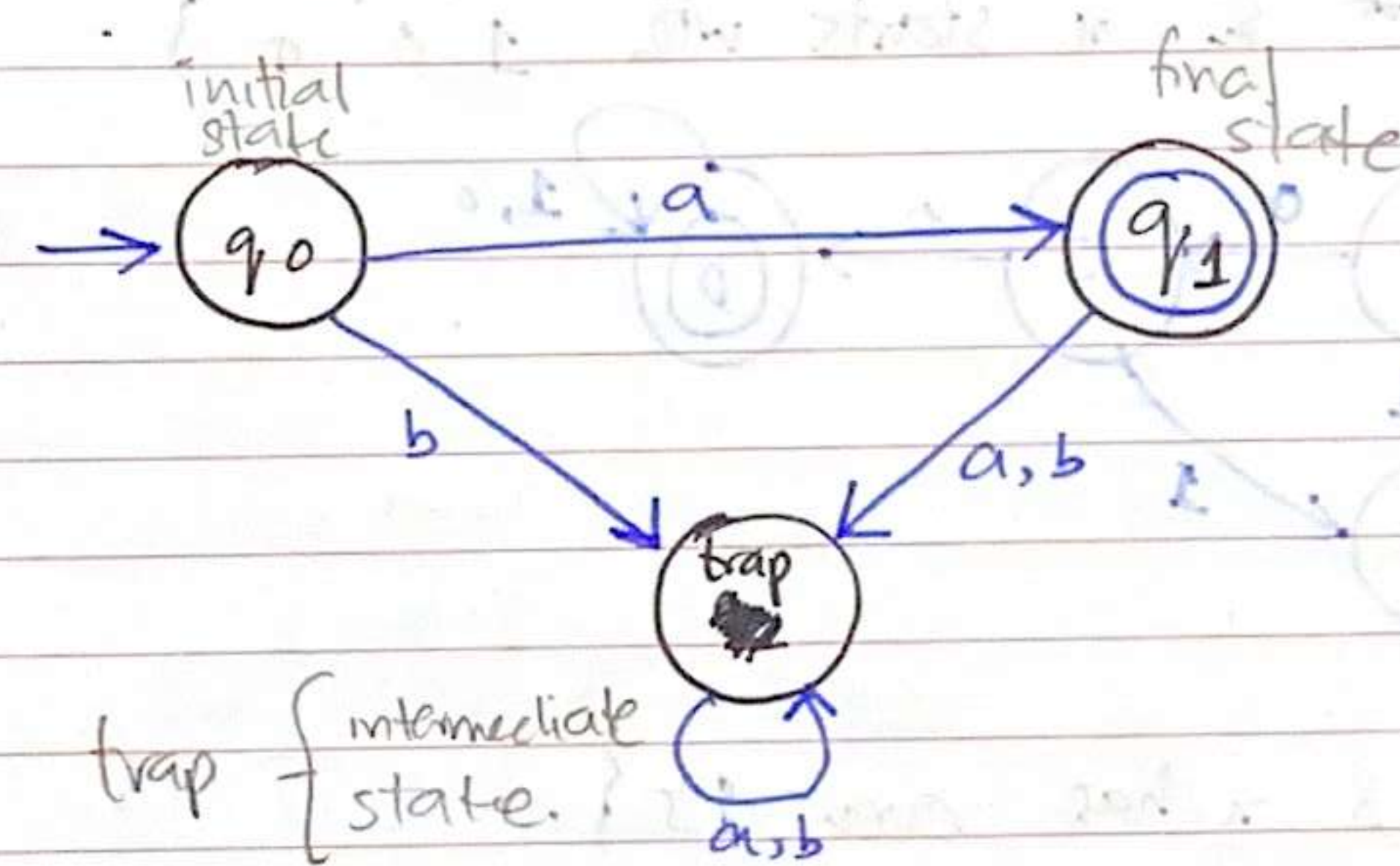
set of states
input set

set of initial states

set of final states.

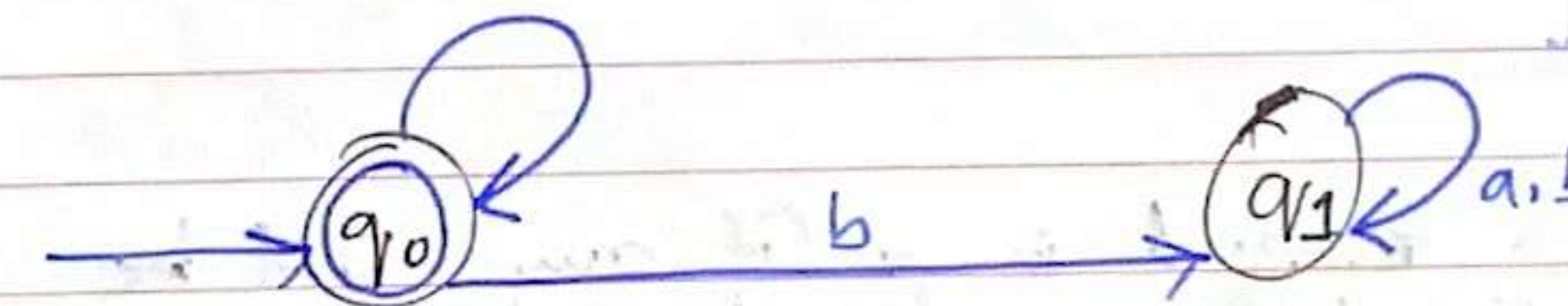
set of transition functions.

$\Rightarrow$ Make state machine for L_3 .
 $L_3 = \{x \mid x \in \Sigma^* \text{ \& } x = a^*\}$
 $L_3 = \{a^*\}$



In trap state, any input will make the machine pop on trap since we can't go any further from a 'trap'.

$\Rightarrow$ State machine for $L_1 = \{x \mid x \in \Sigma^* \text{ \& } x = a^*\}$



* The language that accepts '1' (NULL) will have same initial & final state (But don't write 1 in input)

day / date:

⇒ Transition Table of L_7 :

State \ input	a	b
q_0	q_0	q_1
q_1	q_1	q_1

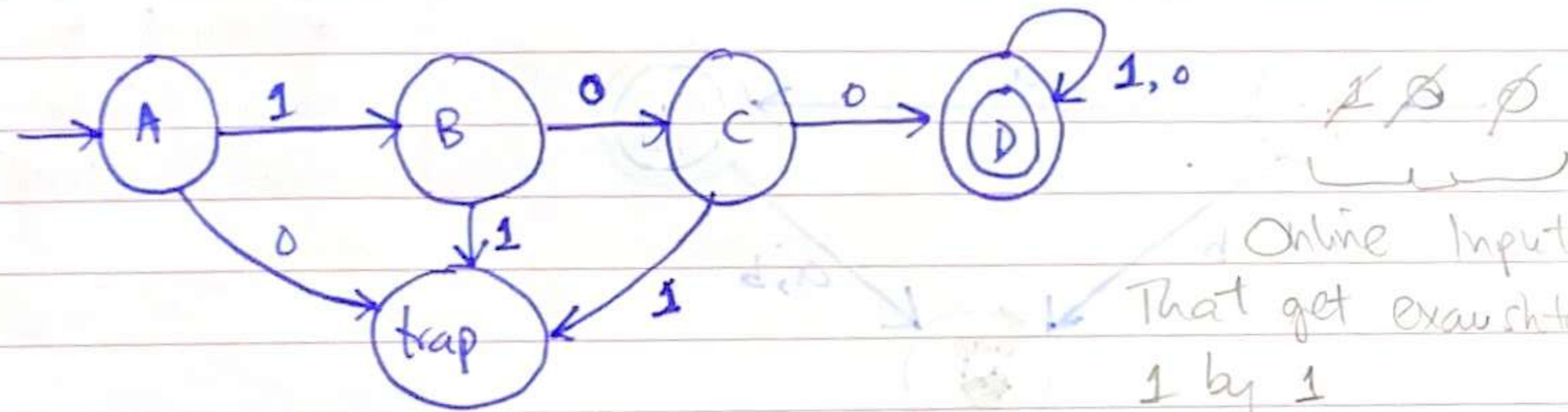
Chap #3 for
initial & final
state in table
from #, -

$\Sigma = \{0, 1\}$.

LECTURE - 3

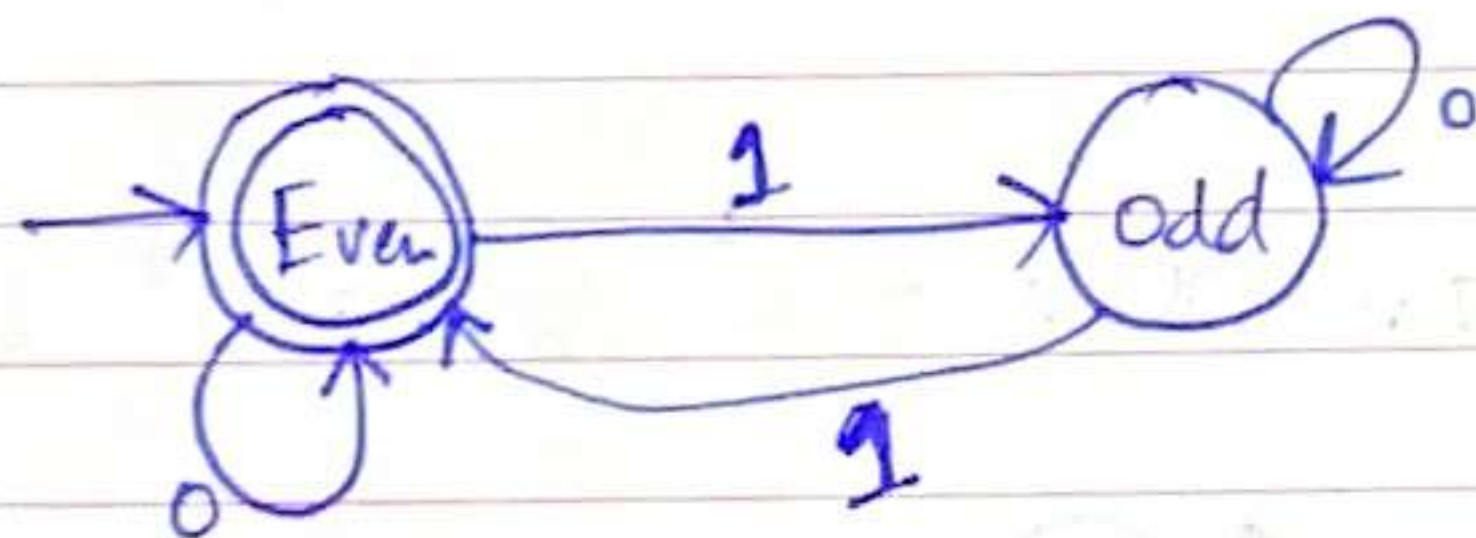
Tuesday
26/8/25

① $L_1 = \{u \mid u \in \Sigma^* \text{ \& } u \text{ starts with } 100\}$.

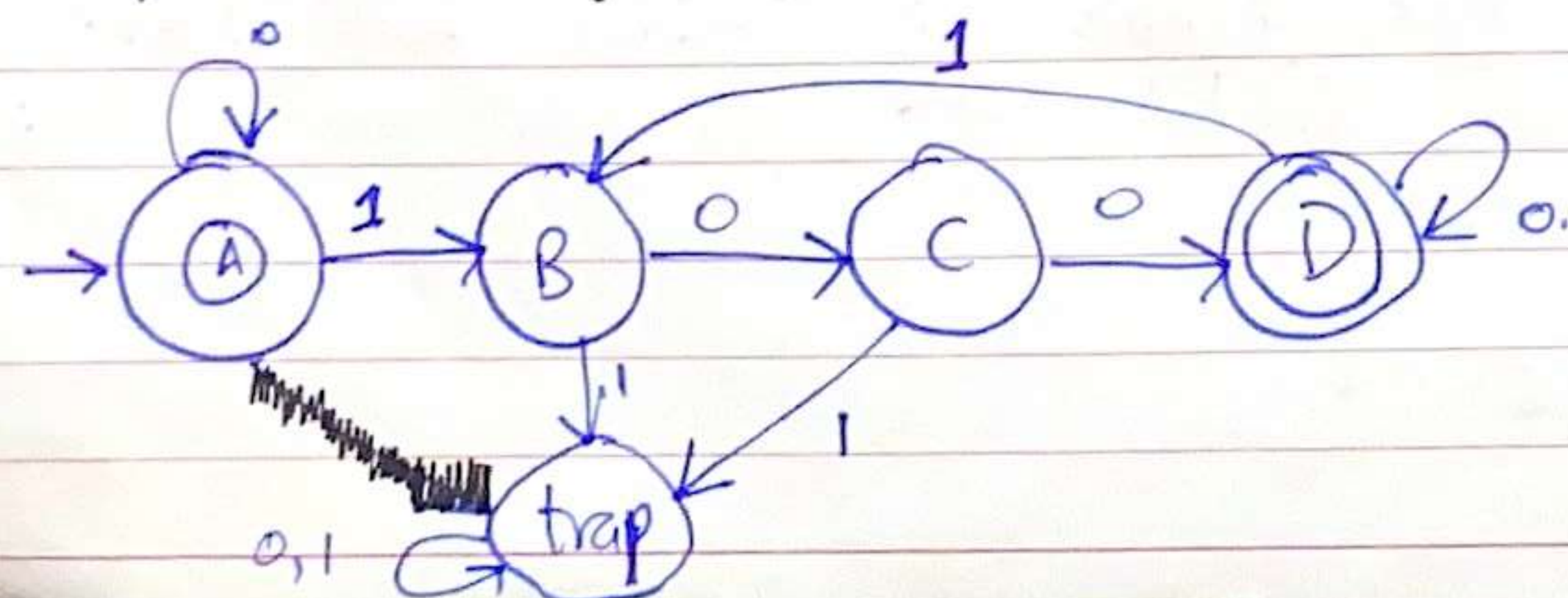


Online Input:
That get exhausted
1 by 1

② $L_2 = \{u \mid u \in \Sigma^* \text{ \& } u \text{ has even 1's}\}$.

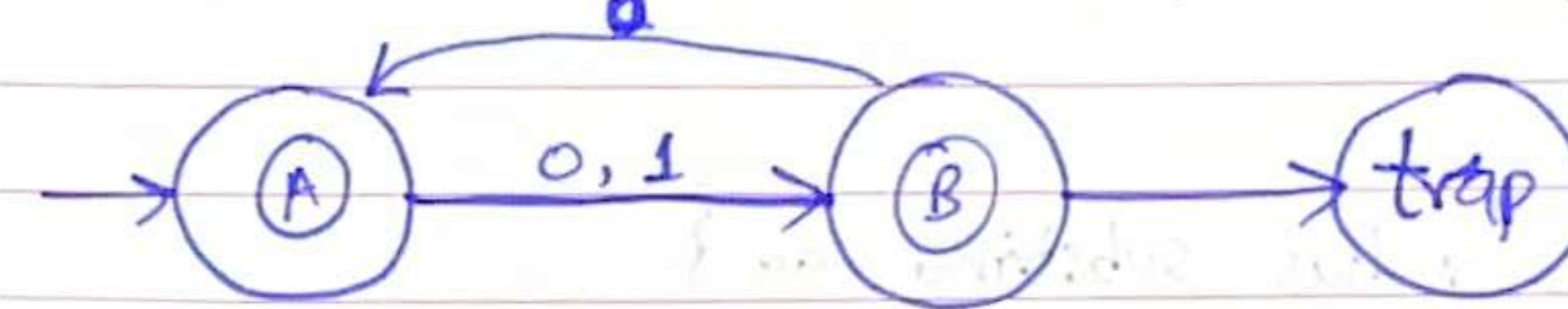


③ $L_3 = \{u \mid u \in \Sigma^* \text{ \& } \text{every } 1 \text{ in } u \text{ (if any) shld be followed by atleast two } 0\}$.

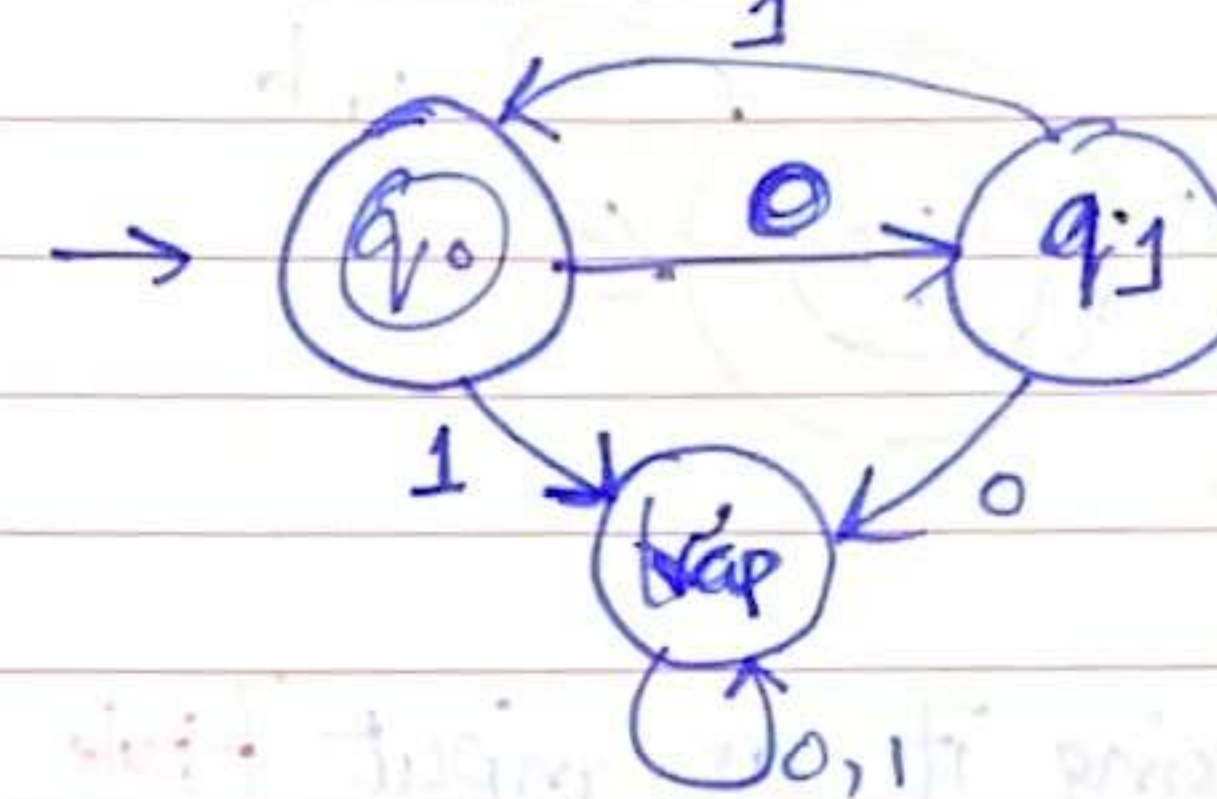


day / date:

④ $L_4 = \{u \mid u \in \Sigma^* \text{ \& } \text{every second element (if any) should be } 0\}$.



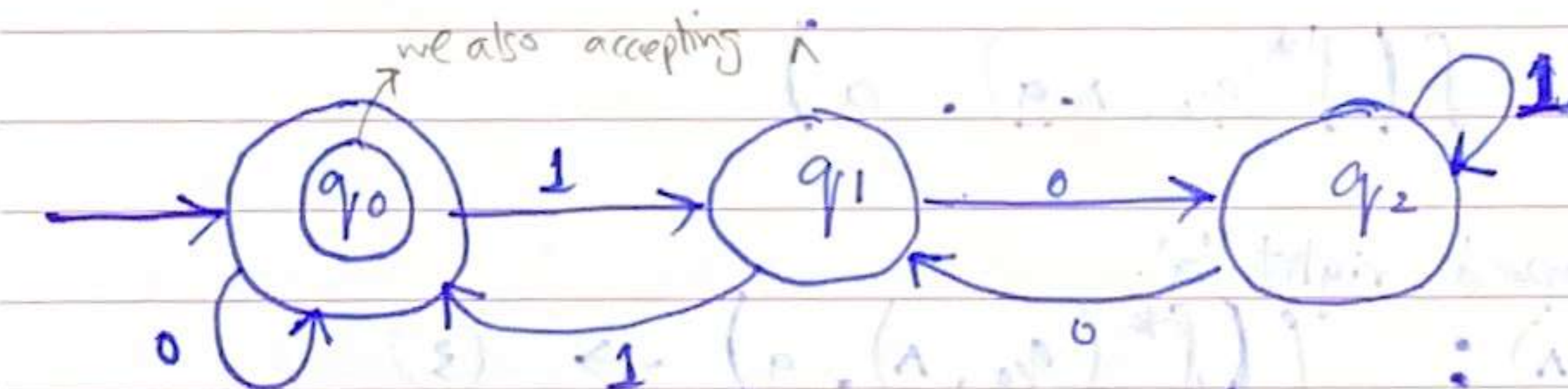
⑤ $L_5 = \{u \mid u \in \Sigma^* \text{ \& } u = (01)^*\}$.



LECTURE - 4

Thursday
28/8/25

$L_1 = \{u \mid u \in \Sigma^* \text{ \& } \text{Binary of } u \text{ divisible by } 3\}$.
* States = 3 (remainder = 0, 1, 2).



⇒ Extended Functions (f^*): To prove if a string input is a part of language or not.

$$① f^*(q, \epsilon) \rightarrow q$$

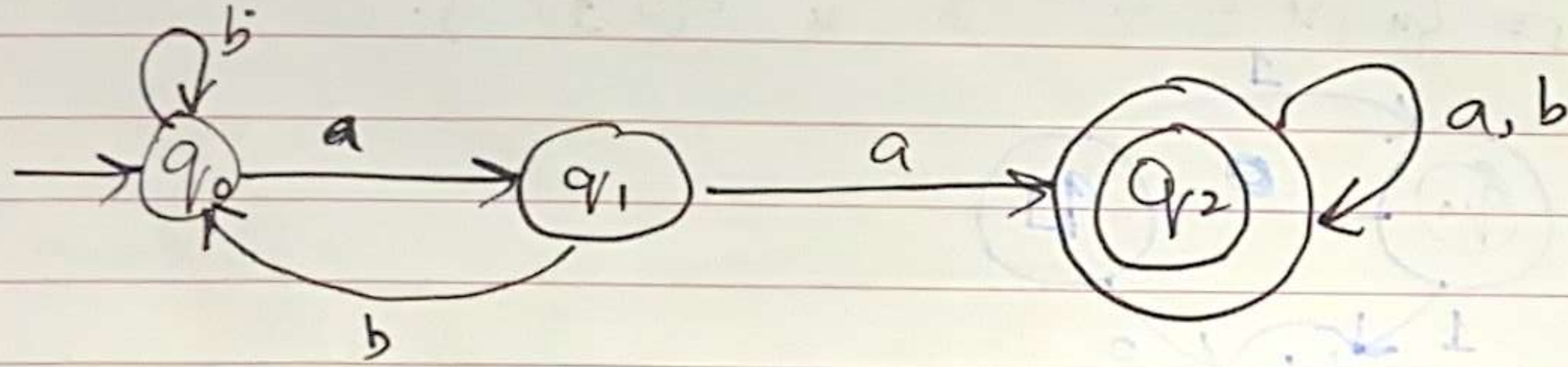
$$② f^*(q, ya) \rightarrow f(f^*(q, y), a)$$

$$③ f^*(q, uz) \rightarrow f^*(f^*(q, u), z)$$

Explanation 3
pages ahead

→ Solving (Removing) δ^* (extended func.) :
 Q1) $\delta^*(q_0, \underline{aab})$ Using Rule 1 & 2

$L = \{w \mid w \in \Sigma^* \text{ \& } w \text{ has substring } aab\}$
 $Q1 = \{q_0, q_1, q_2\}$



★ chopping right most element (b) and using it as input } Rule 2

$$\delta(\delta^*(q_0, \underline{aa}), b) \rightarrow \textcircled{1}$$

★ Again chopping right most element } Rule 2.

$$\delta^*(q_0, aa) : \delta(\delta^*(q_0, a), a) \rightarrow \textcircled{2}$$

★ $\Lambda \cdot a = a$ (concatenate) $\therefore \delta(\delta^*(q_0, \Lambda \cdot \underline{a}), a)$

★ Again chopped right 'a':
 $\delta^*(q_0, a \cdot \Lambda) : \delta(\delta^*(q_0, \Lambda), a) \rightarrow \textcircled{3}$

Rule 1
 $\delta^*(q_0, \Lambda) \rightarrow q_0$

★ Now since we've an answer (next state) i.e. q_0 , therefore we can back substitute this value into eq ③ → eq ② → eq ① to check if the next state after eq ① is the final state in diagram, then 'aab' is part of language.

★ Putting ' q_0 ' in eq ③:

$$\delta^*(q_0, \Lambda) = q_0$$

$$\therefore \delta(\delta^*(q_0, \Lambda), a) \rightarrow \delta(q_0, a)$$

★ From the diagram: If input at ' q_0 ' is 'a', next state = q_1

→ Now substitute this in eq ②.

using diagram
 $\delta(q_1, a) \rightarrow q_2$

→ Now substitute this in eq ①.

using diagram
 $\delta(q_2, b) \rightarrow q_2$

∴ this is the next state from eq ① & since ' q_2 ' is final state in question's diagram ∴ 'aab' string is part of language.

Q2) $\delta^*(q_0, \underline{aabab})$: Rule 3

★ Since we solve a string inside "aabab" which was 'aab' in Q1; therefore we can substitute the answer (next state) of aab (which was q_2) here.
 ★ The only catch is that already solved string should be on left most position of the new string.

NOTE : $\delta^*(q_0, ab \underline{aab})$

cannot substitute ' q_2 ' here using Rule 3 since aab (already solved string is on right most).

Explanation of Extended Func.

- ① $\delta^*(q, \Lambda) \rightarrow q$: Inputting Λ will result in same state as initial i.e. q . day / date:
- ② $\delta^*(q, ya) \rightarrow \delta^*(\delta^*(q, y), a)$: If input is of string more than 1 character then we'll chop the right most element and put it as input of a simple func. The rest part of the string will be rewritten and resolved using rule ② until we've a single character string in input. When we get single character, we'll find next state from diagram in q/s and unwind like in recursion to put next state into prev. ex.
- ③ $\delta^*(q, xz) \rightarrow \delta^*(\delta^*(q, x), z)$: If we've a string whose 'x' part is already solved & we've the next state answer, then substitute that part & solve the rest using rule ②. The already solve part should be on left of the string that is to be solve using ②

$$L_1 = \{x \mid x \in \Sigma^* \text{ \& } x \text{ has substring } aa\}$$

$$L_2 = \{x \mid x \in \Sigma^* \text{ \& } x \text{ ends with } ba\}$$

(a) $L_3 = L_1 \cup L_2 \rightarrow \{x \mid x \in \Sigma^* \text{ \& } x \text{ has a string } aa \text{ or } x \text{ ends with } ba\}$

$$L_3 = \{aa, ba, baa, aaa, bba, aba, aab, \dots\}$$

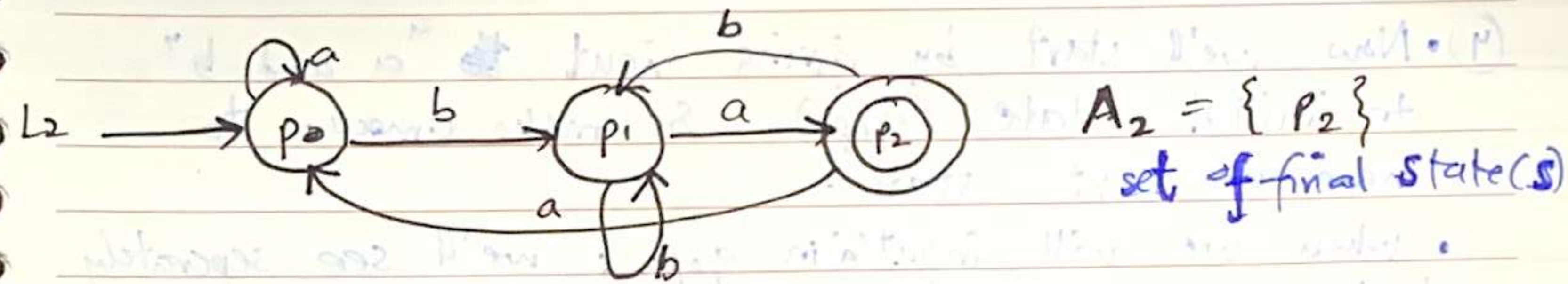
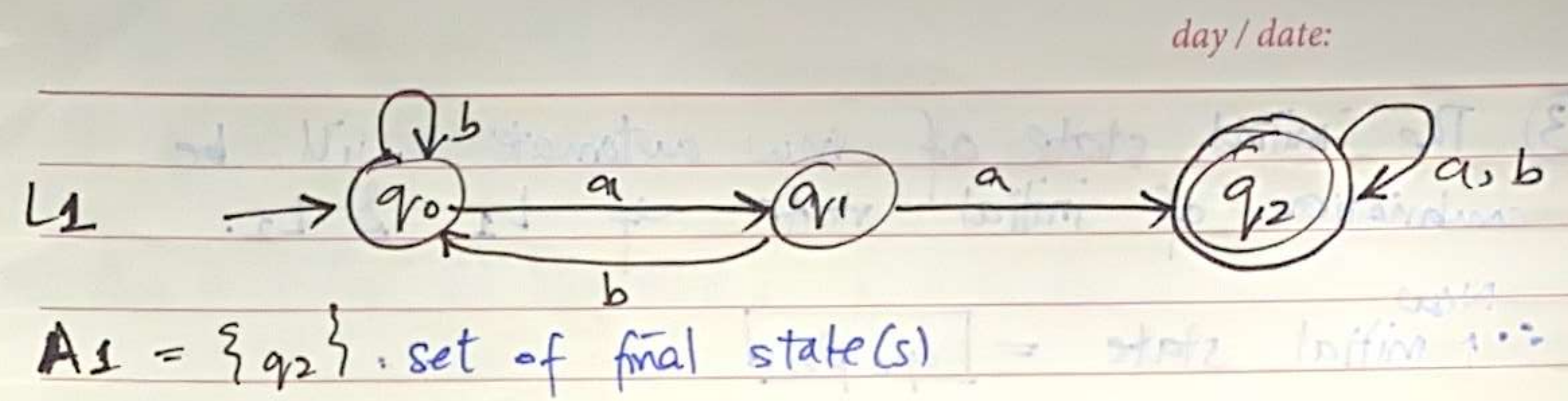
(b) $L_4 = L_1 \cap L_2 \rightarrow \{x \mid x \in \Sigma^* \text{ \& } x \text{ has a string } aa \text{ and } x \text{ ends with } ba\}$

$$L_4 = \{aaba, baaba, aabba, \dots\}$$

(c) $L_5 = L_1 - L_2 \rightarrow \{x \mid x \in \Sigma^* \text{ \& } x \text{ has a string } aa \text{ and does not ends with } ba\}$

$$L_5 = \{aa, aaa, aab, baa, abaa, \dots\}$$

★ Since we have developed new languages (L_3, L_4, L_5) $\therefore$ new automata machine will be made. BUT we can make new automate using state machines of L_1 & L_2 (in Ques.).



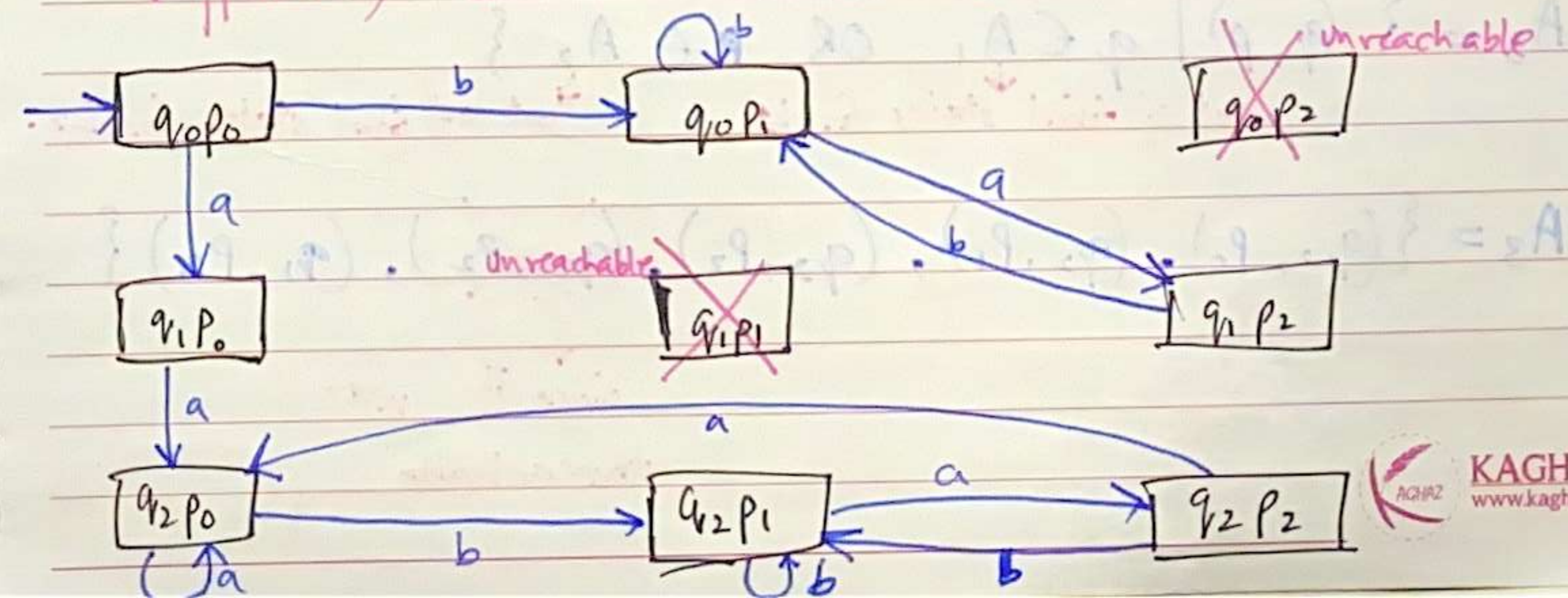
① Change the names of states in either language's diagram if their names are same.

② Take cartesian product.

$$Q_1 \times Q_2 = \{q_0, q_1, q_2\} \times \{p_0, p_1, p_2\}$$

$$= \{(q_0, p_0), (q_0, p_1), (q_0, p_2), (q_1, p_0), (q_1, p_1), (q_1, p_2), (q_2, p_0), (q_2, p_1), (q_2, p_2)\}$$

★ Since we've 9 possible combinations of $Q_1 \times Q_2$ $\therefore$ the no. of states in new automata (union, intersection, difference) can not exceed '9'.



- ③ The initial state of new automate will be combination of initial state of L_1 & L_2 .

• New initial state = $q_0 p_0$

- ④ Now we'll start by giving input "a and b" to initial state ($q_0 p_0$) & make arrows to move to next state.

• When we will input 'a' in $q_0 p_0$; we'll see separately that q_0 par input 'a' anay pr what's next state & then p_0 par 'a' anay pr what's the next state.

• $\left. \begin{array}{l} \delta(q_0, a) \rightarrow q_1 \\ \delta(p_0, a) \rightarrow p_0 \end{array} \right\} \therefore \text{next state from "q_0 p_0" on input a} = q_1 p_0$

$\delta(q_0 p_0, a) \rightarrow q_1 p_0$

- ⑤ Complete the diagram using step 4 repetitively.

- ⑥ Cancel out Unreachable states

- ⑦ Define final state. If either final state (from L_1 or L_2) is reached, then the final state of new machine would be reached -

$$A_3 = \{ (q, p) \mid q \in A_1 \text{ OR } p \in A_2 \}$$

set of final states for L_1

set of final states for L_2

$$A_3 = \{ (q_2, p_0), (q_2, p_1), (q_2, p_2), (q_0, p_2), (q_1, p_2) \}$$

since it was unreachable

- ⑧ Final state for " $L_4 = L_1 \cap L_2$ "

$$A_4 = \{ (q, p) \mid q \in A_1 \text{ & } p \in A_2 \}$$

$$A_4 = \{ q_2, p_2 \}$$

- ⑨ Final state for " $L_5 = L_1 - L_2$ "

$$A_5 = \{ (q, p) \mid q \in A_1 \text{ & } p \notin A_2 \}$$

$$A_5 = \{ (q_2, p_0), (q_2, p_1), \cancel{(q_2, p_2)} \}$$

LECTURE-5

Tuesday
2/9/25

- Regular language: DFA exist & Regular expression
• Non Regular lang: DFA doesn't exist

eg: $L_1 = \{ a^n b^n \mid n > 0 \}$

⇒ Deriving Regular Expression from "Language"

Regular expression: A specific formula through which we can generate every element in our language.

→ Formation:

already ← Language

enumerated

$\{ \Lambda \}$

$\{ a \}$

$\{ ab, a \}$

only single

'a' or single 'b'

$\{ a, b \}$

$\{ a, aa, aaa, \dots, a^5 \}$

$\{ \Lambda, a, aa, aaa, \dots, a^5 \}$

infinite

$$(d+d) = (d+d)(d+d)(d+d)(d+d)$$

★ For ↑ loop, use kleene star or +

For finite loop, use finite numbered power.

start with the first (smallest) element of language.

Reg. Expression

Λ

a

ab + a OR a + ab

a + b

a^+

a^*

first time old be 'a'
 second time old be 'b'
 third time old be 'a' } ∴ aba

day / date:

$$(a+b)^* = (1+a+b)(1+a+b)(1+a+b) \dots$$

$$\therefore \{ \Lambda, a, b, aa, bb, ab, ba, \dots \}$$

$$L_3 = L_1 \cup L_2$$

Reg. expression of $L_1 : r_1$
 " " " $L_2 : r_2$
 " " " $L_3 : r_1 \cup r_2$

Q: Reg. Expression of following.

(a) $L_1 = \{u \mid u \in \Sigma^* \text{ starts with } ab\}$

$$L_1 = \{ab, aab, bab, \dots\}$$

$$\therefore \boxed{ab(a+b)^*}$$

order of letter matters

(b) $L_2 = \{u \mid u \in \Sigma^* \text{ end with } ab\}$

$$(a+b)^* ab$$

(c) $L_3 = \{u \mid u \in \Sigma^* \text{ has substring } ab\}$

$$(a+b)^* ab (a+b)^*$$

(d) $L_4 = \{u \mid u \text{ has at least 3 a's}\}$

$$b^* a b^* a b^* a (a+b)^*$$

(e) $L_5 = \{n_a(u) \text{ are multiple of 4}\}$

$$(b^* a b^* a b^* a b^* a b^*)^* \cdot b^*$$

(f) $L_6 = \{n_a(u) \text{ are even}\}$

$$(b^* a b^* a b^*)^* \cdot b^*$$

can not be $(a^* + a^*) \cdot b^*$ as $(a^* + a^*) = a^*$

(g) $L_7 = \{u \mid \text{length of } u \text{ is } 4\}$

$$(a+b)(a+b)(a+b)(a+b) = (a+b)^4$$

$$(r_1 + r_2)^* \neq r_1^* + r_2^*$$

day / date:

Q: Set of language from Reg. expression.

(a) $b^* a b^* = \{a, ba, ab, bab, bbab, babb, \dots\}$

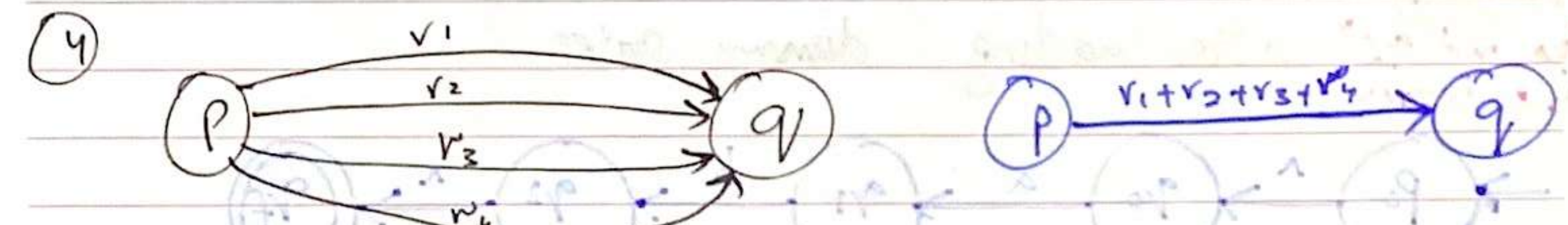
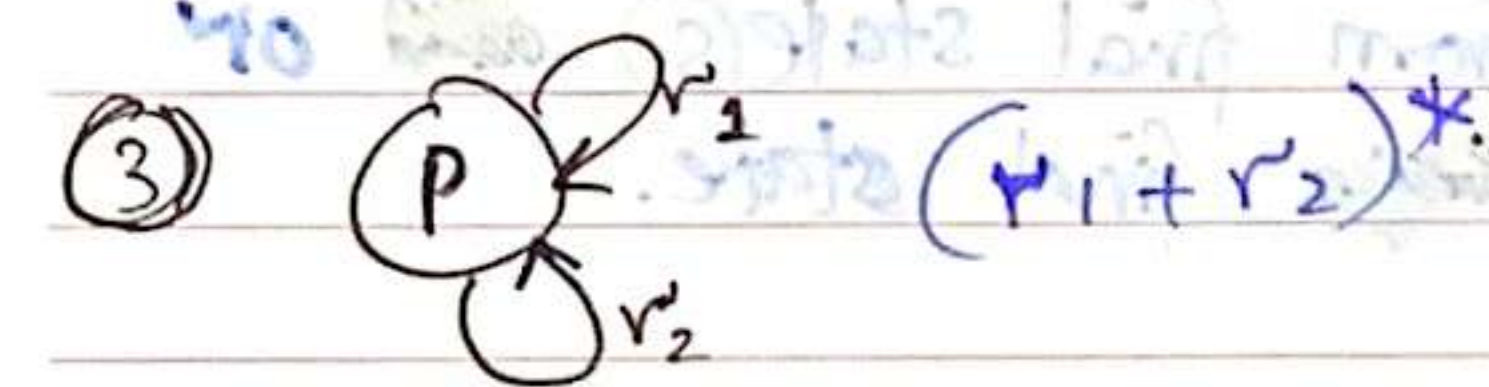
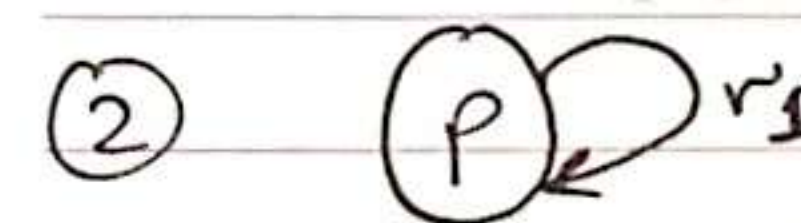
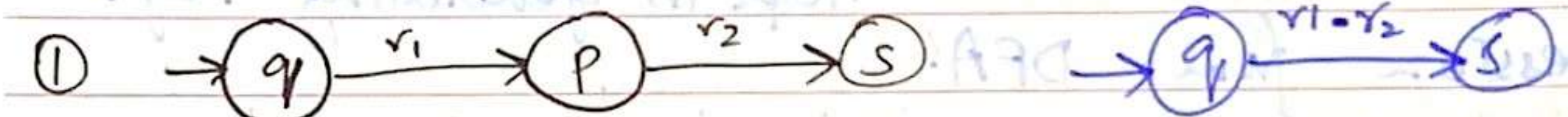
(b) $b(a+b)^* a = \{ba, baa, bba, \dots\}$

(c) $(a^* + b^*) = \{\Lambda, a, aa, aaa, b, bb, bbb, \dots\}$

(d) $(a^* + b^*)^* = (a+b)^* = \text{every possible combo.}$

(e) $(a^*)^* = a^*$

State Elimination.

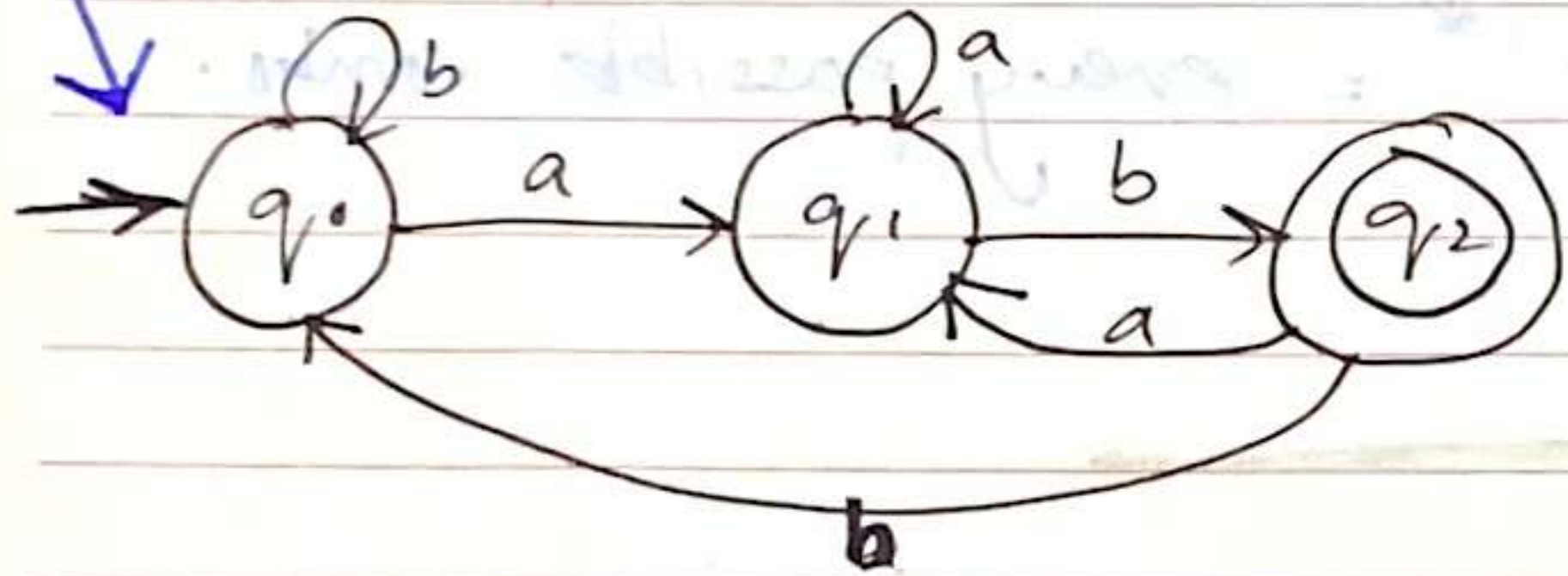


* If more than one arcs are going from one state to another, then we'll write $__ + __$ in reg. expression

If one arc is followed by another like $q_0 \xrightarrow{a} q_1 \xrightarrow{b} q_2$, then we will concatenate like $a \cdot b$

Deriving Reg-Expression from "Automata" (DFA):

★ Deriving reg. expression from Automata is more reliable than deriving it from language.
e.g: case of $L \cap L^c$ have more chances of mistakes.



- Making Dummy States: Helps in generating reg. expression from DFA.

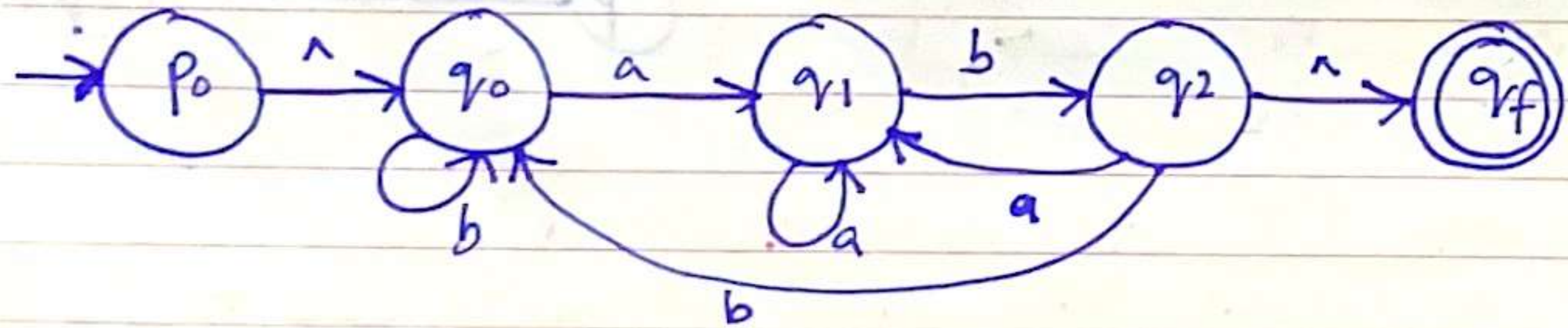
⇒ Make Dummy initial state when:

~~incoming~~ incoming input on initial

⇒ Make Dummy final state when:

There is outgoing arcs from final state(s) ~~or~~ or
 there ~~be~~ ~~be~~ ~~be~~ ~~be~~ final state.
 multiple

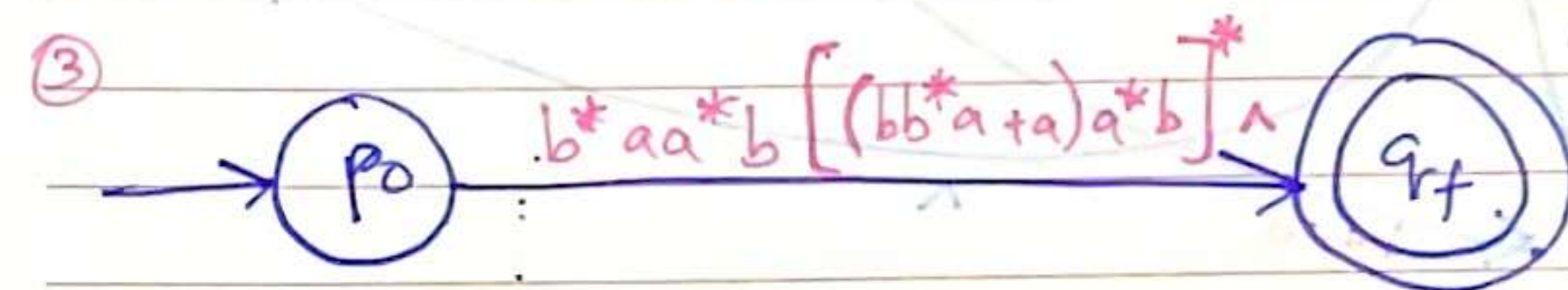
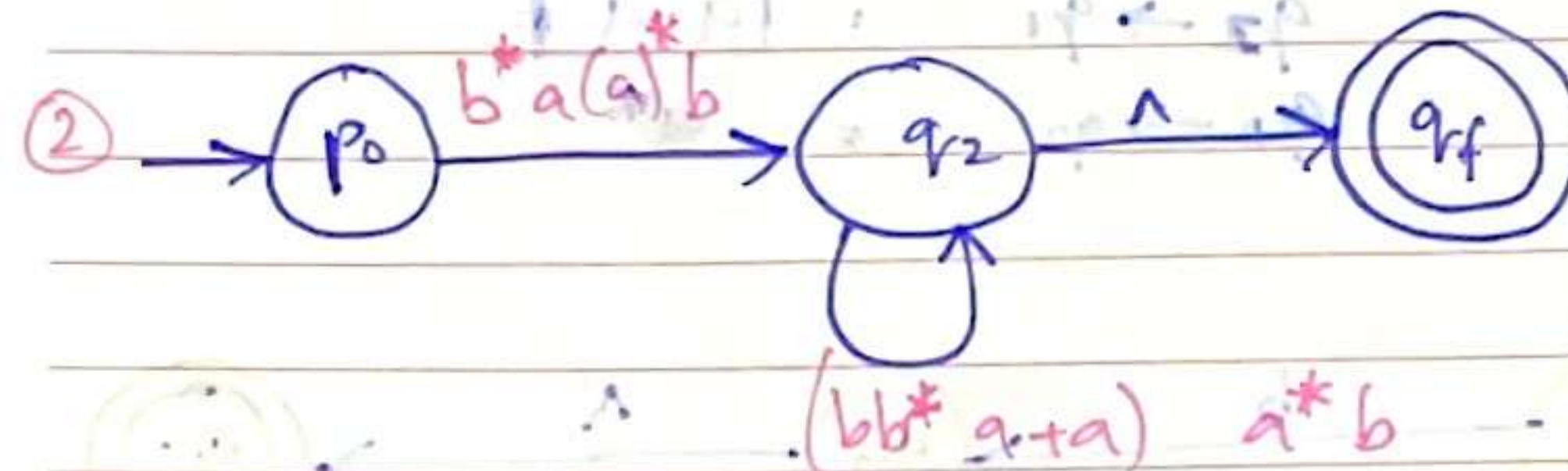
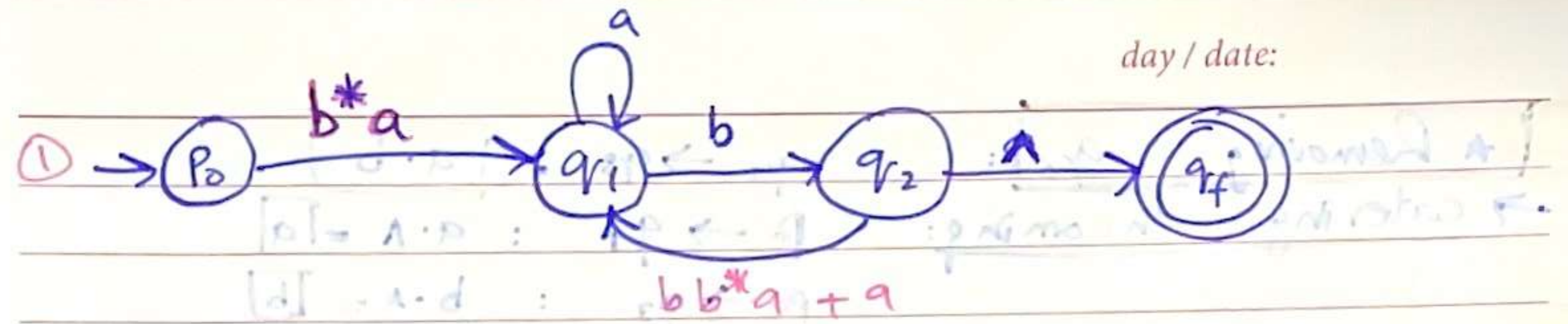
~~AIM~~ ∴ The following is the DFA of above machine "p0 and qf" after adding dummy states.
are dummy



AIM: We have to remove all middle states and only existing states should be $(p_0 \rightarrow q_f)$. The expression made while doing this will be our **REG. EXPRESSION**.

KAGHAZ
www.kaghaz.pk

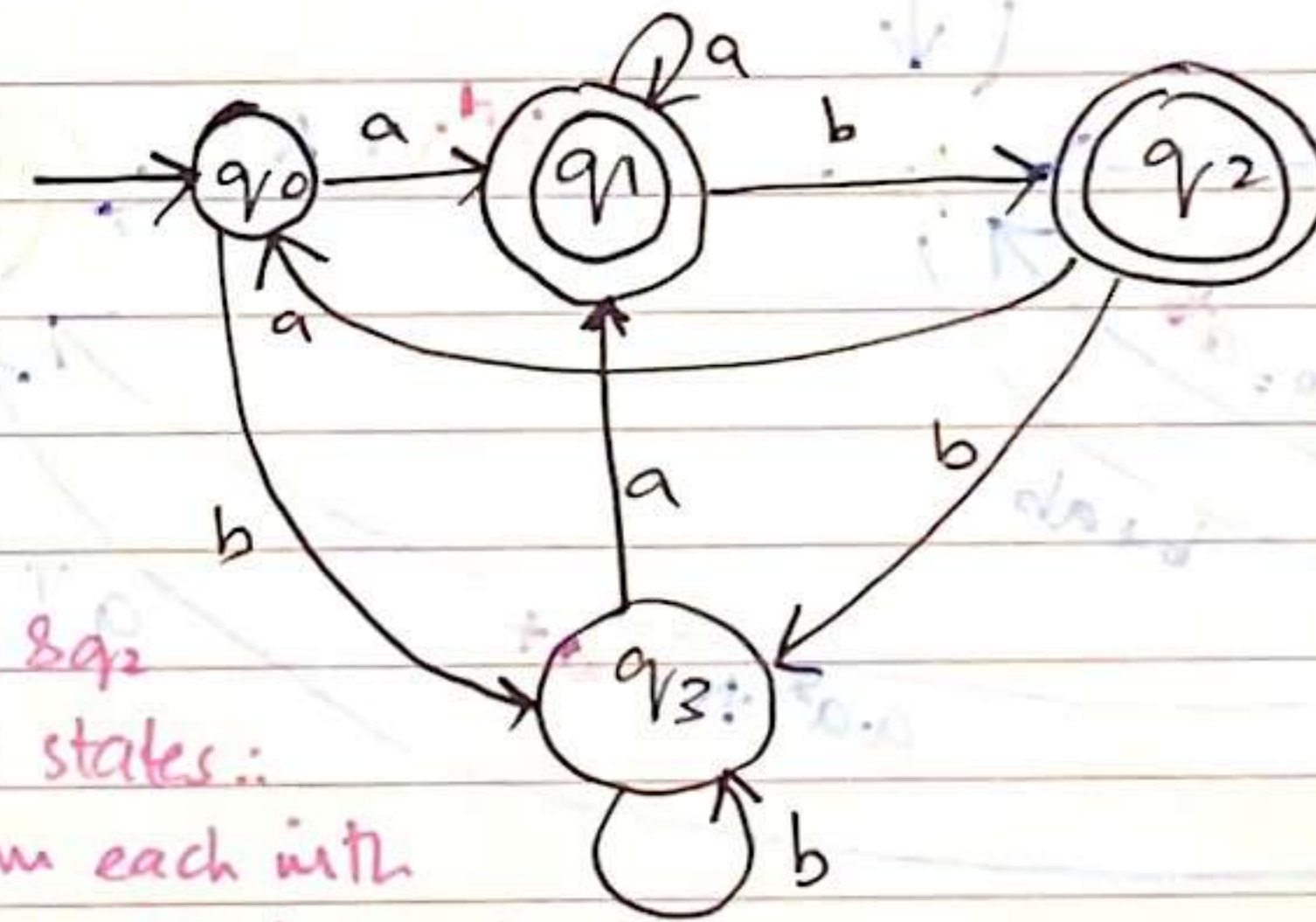
day / date:



∴ This is the regular expression made with DFA which is more reliable.

LECTURE - 6

Thursday
7/9/25

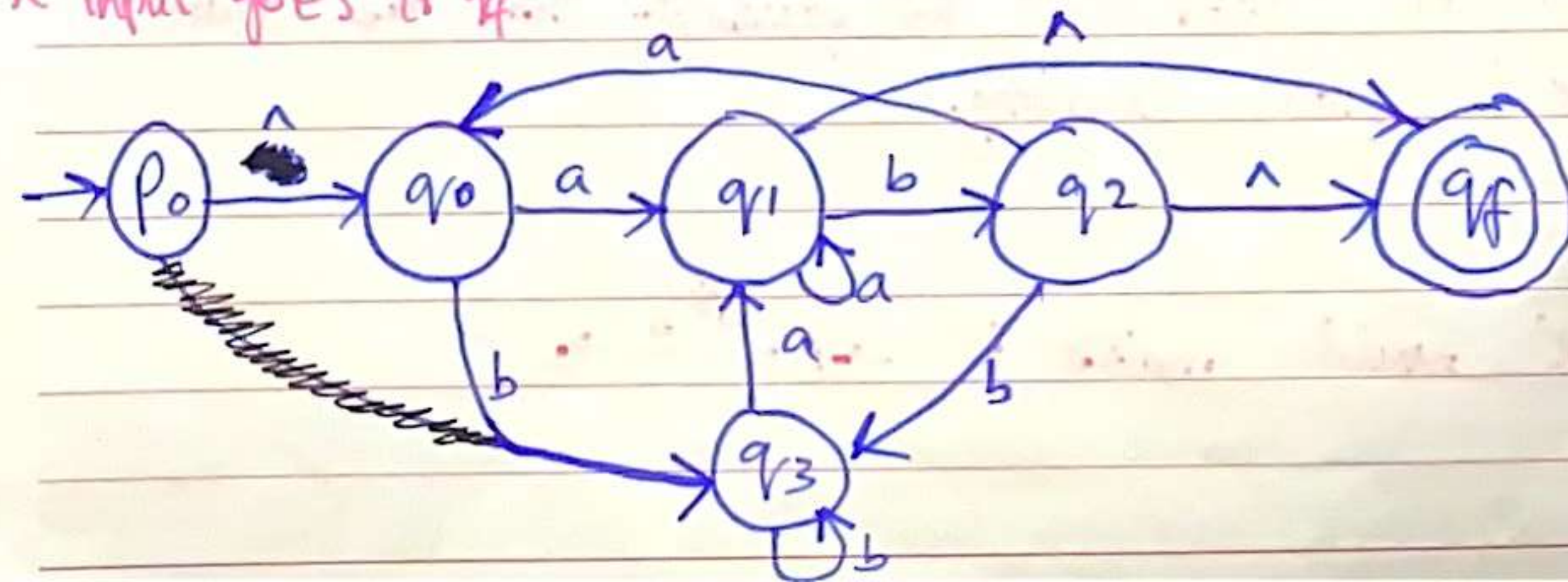


Q: Find Reg. Expression

★ Introduce dummy initial since incoming arc on q_0 .

* Introduce dummy final
since outgoing arc from q_1, q_2
& multiple final states too.

since q_1 & q_2
were final states.:
arrow from each with
an input goes to q_1 .

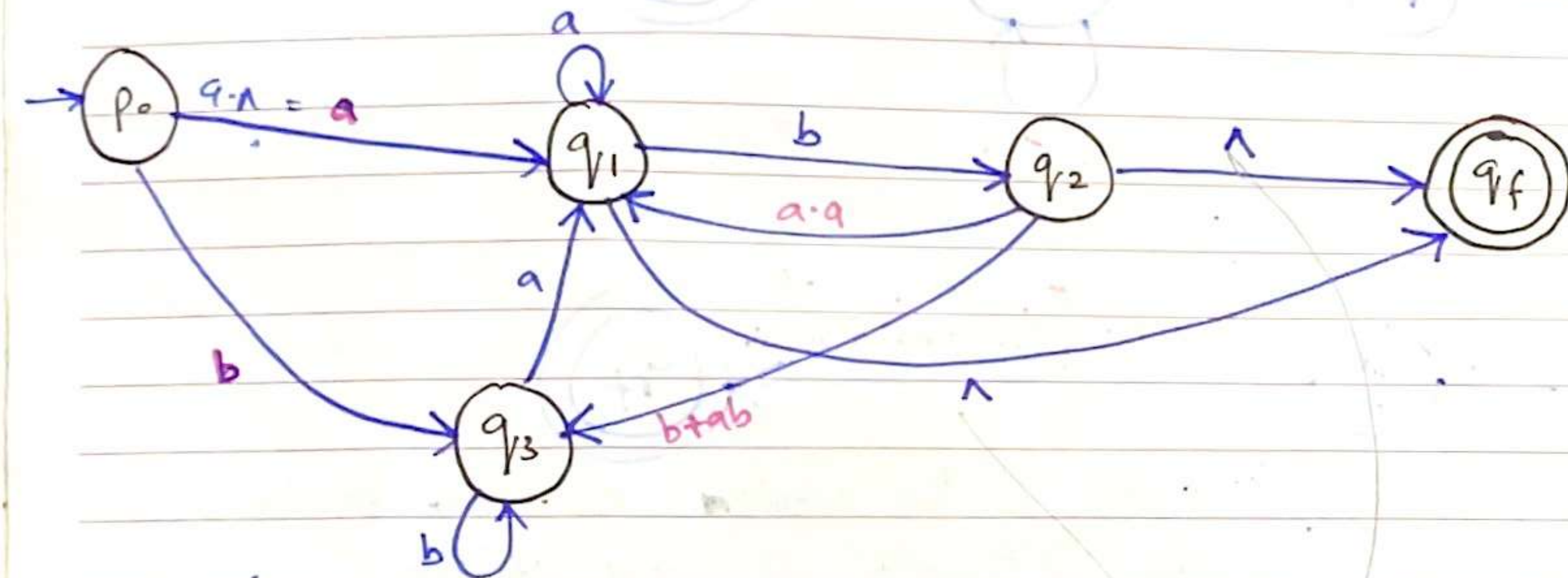


$$a \cdot a^* = a^+$$

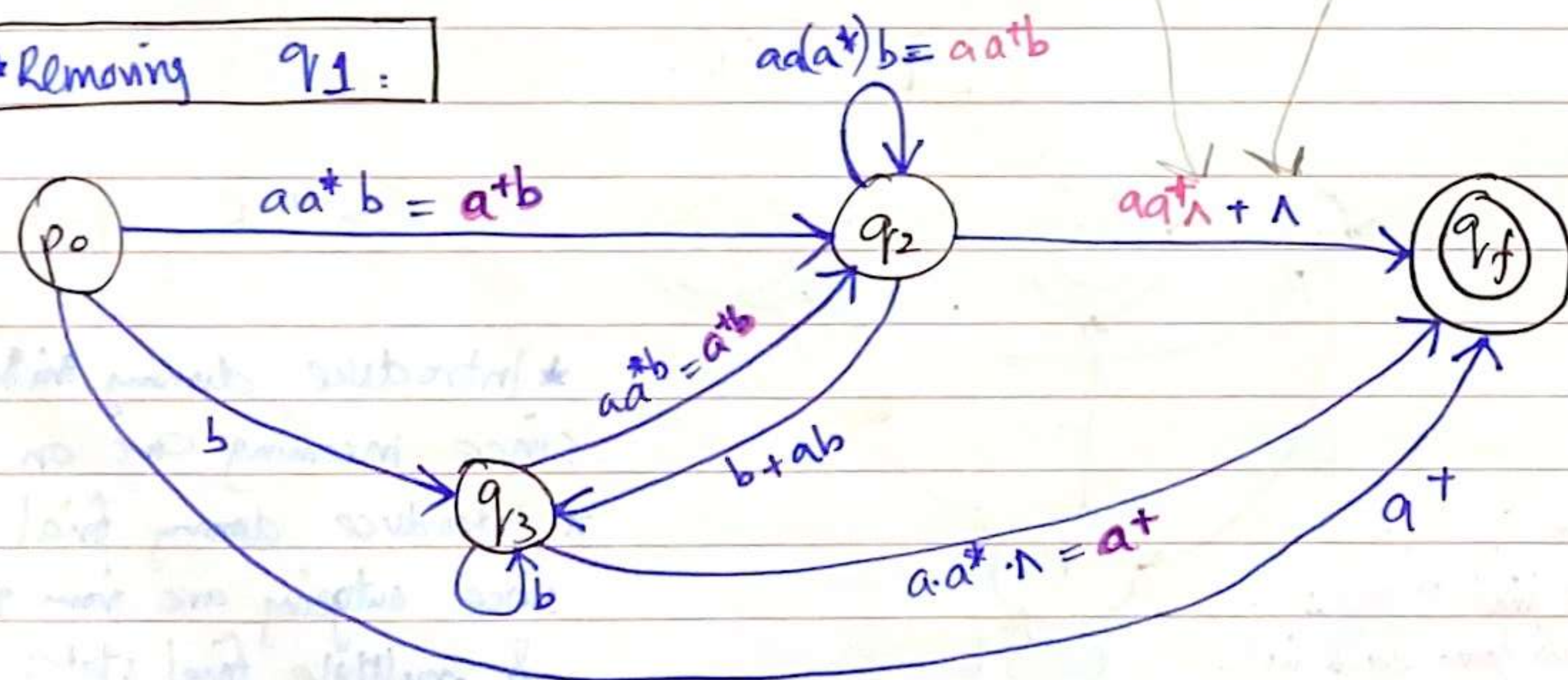
day / date:

★ Removing q_0 :
→ catering Incoming:

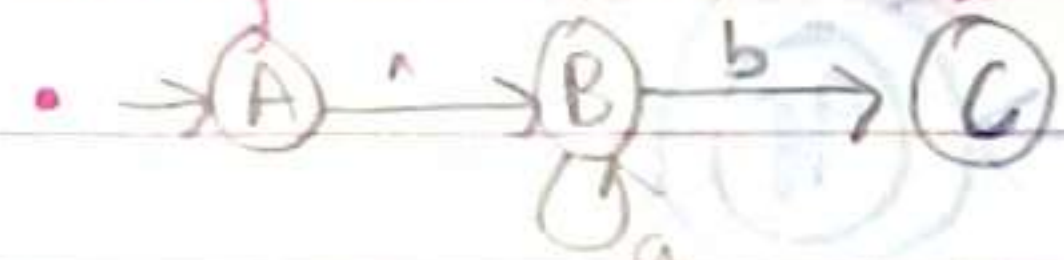
$$\begin{aligned} q_2 \rightarrow q_3 &: a \cdot b \\ p_0 \rightarrow q_1 &: a \cdot \lambda = [a] \\ p_0 \rightarrow q_3 &: b \cdot \lambda = [b] \\ q_2 \rightarrow q_1 &: [a \cdot a] \end{aligned}$$



★ Removing q_1 :



[METHOD] we've to cater all the possibility that the state being removed was fulfilling.

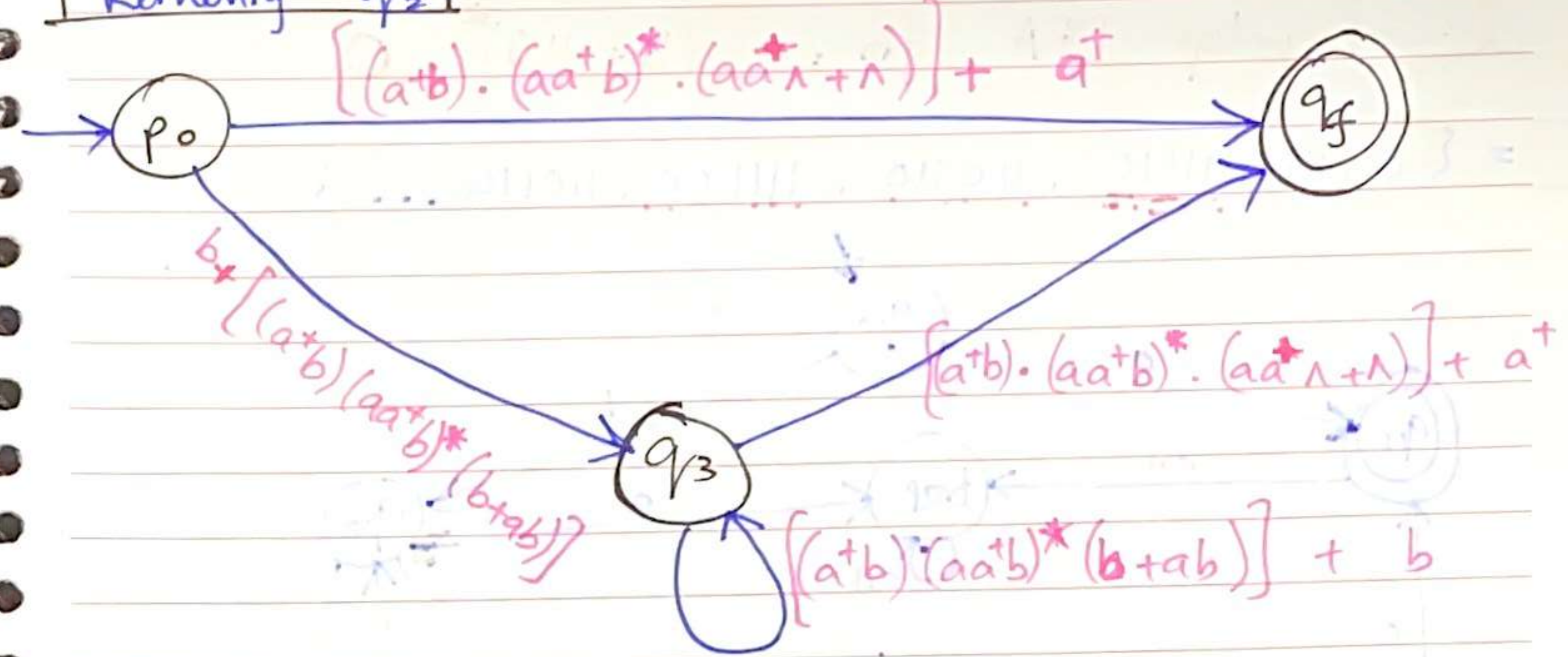


Remaining 'B' would make it $A \xrightarrow{a^*b} C$

when you're passing a state about to be removed with self loop, then Kleene star will definitely have to go to next state.

day / date:

Removing q_2 :

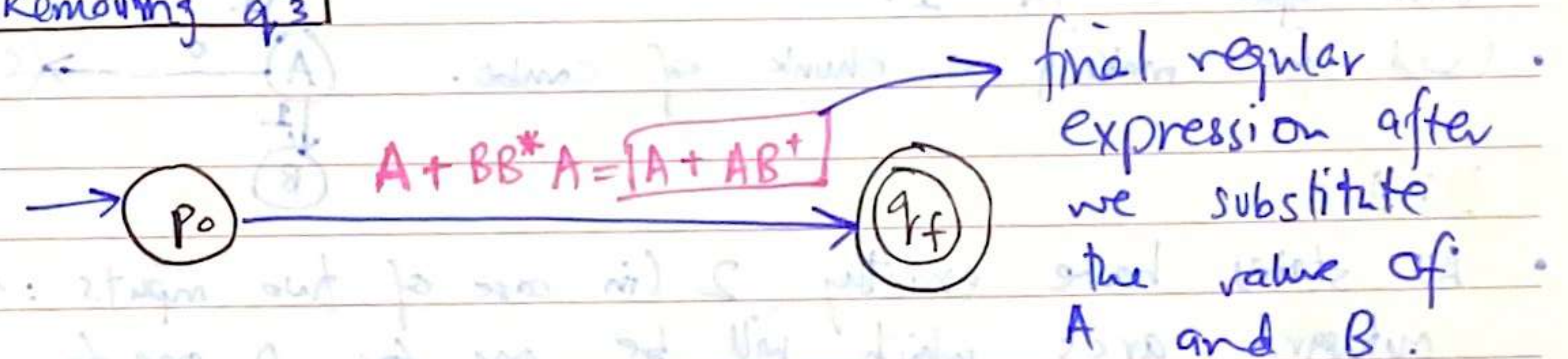


★ Since the expressions are getting very long & there are similar expressions ∴ we'll assign a variable:

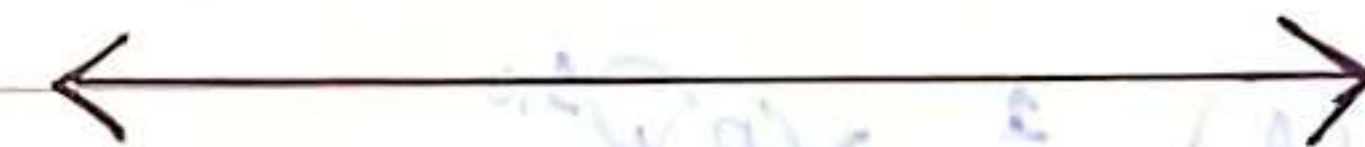
$$\begin{aligned} p_0 \rightarrow q_f \\ q_3 \rightarrow q_f \end{aligned} \} A$$

$$\begin{aligned} p_0 \rightarrow q_3 \\ q_3 \rightarrow q_3 \end{aligned} \} B$$

Removing q_3 :



final regular expression after we substitute the value of A and B.



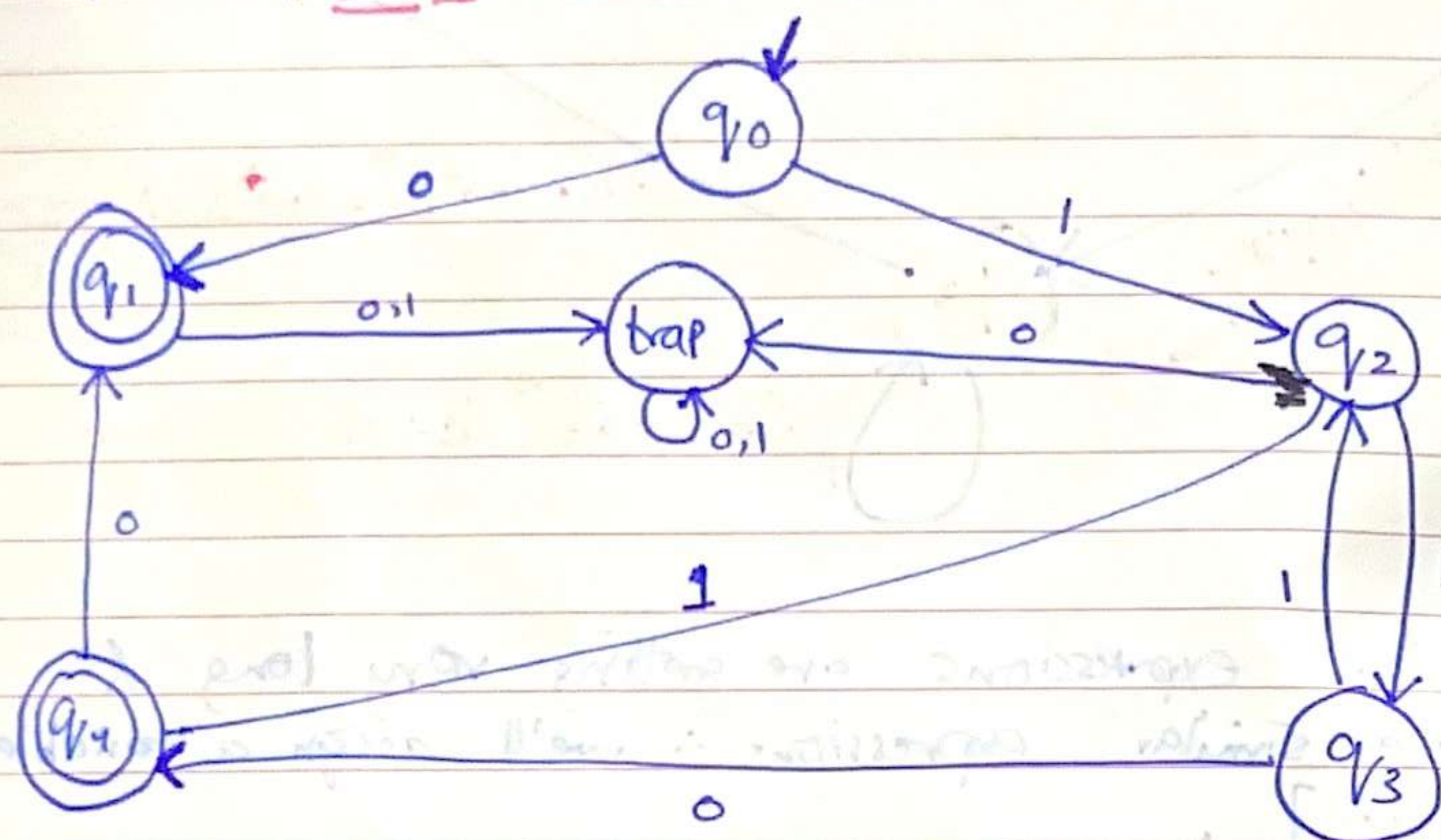
Tuesday
9/9/25

LECTURE-7

day / date:

Q: Develop DFA for: $(11+110)^* 0$

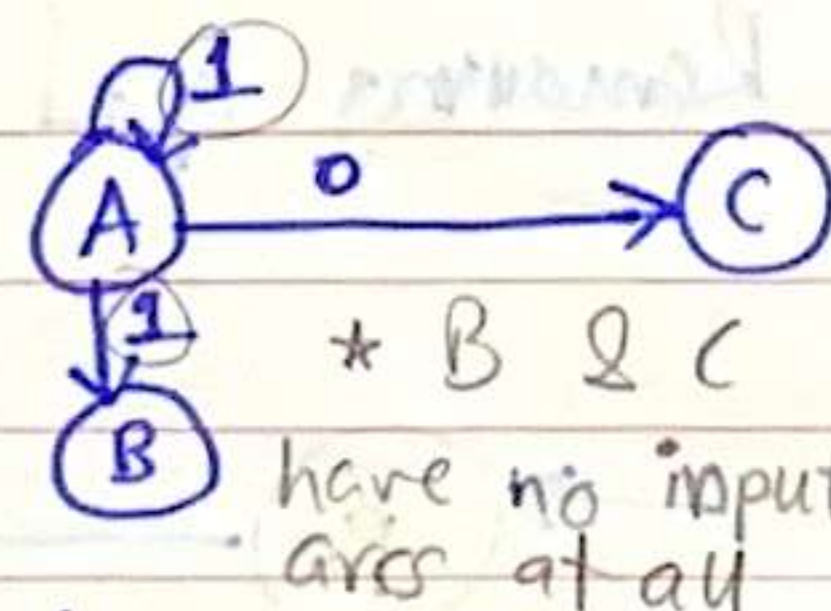
= { 0, 110, 11110, 110110, 111100, 1101100, ... }



Intermediate state which helps to form DFA.

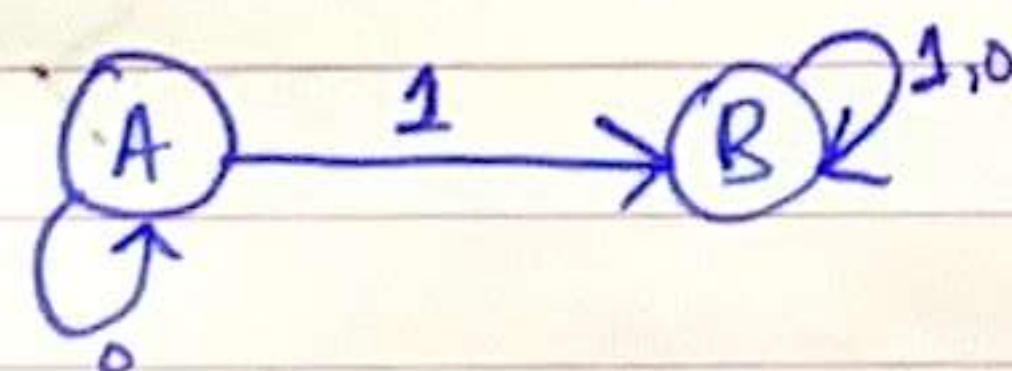
NFA: (Non-deterministic finite Automata)

- Any state can have zero numbers of path as well as multiple paths (max states = no. of states)
- Means a state can have two or more outwards arcs with input 0 or 1.
- Used for making a chunk of combo.



DFA:

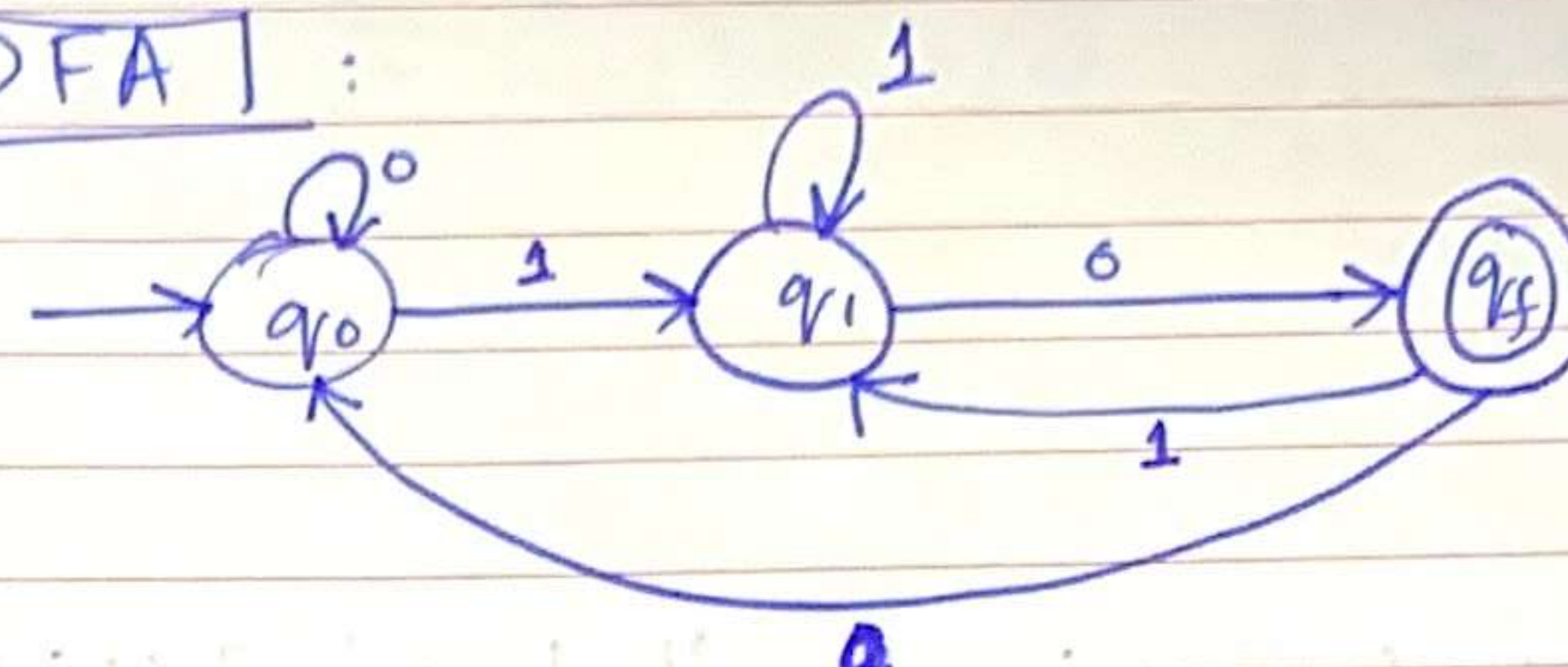
- All states have exactly 2 (in case of two inputs: 0, 1) outward arcs which will be one for 0, one for 1



day / date:

Q: $(0+1)^* 10$

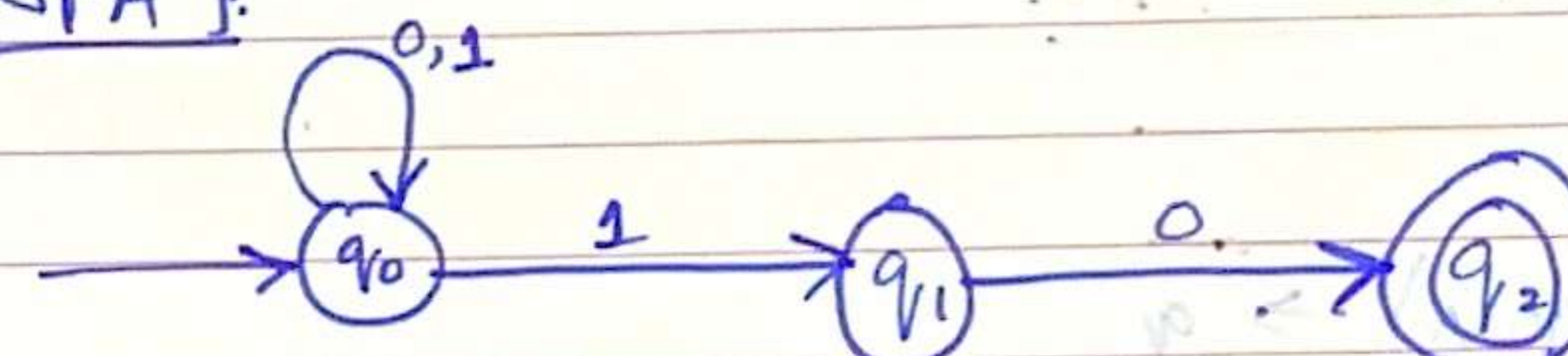
DFA:



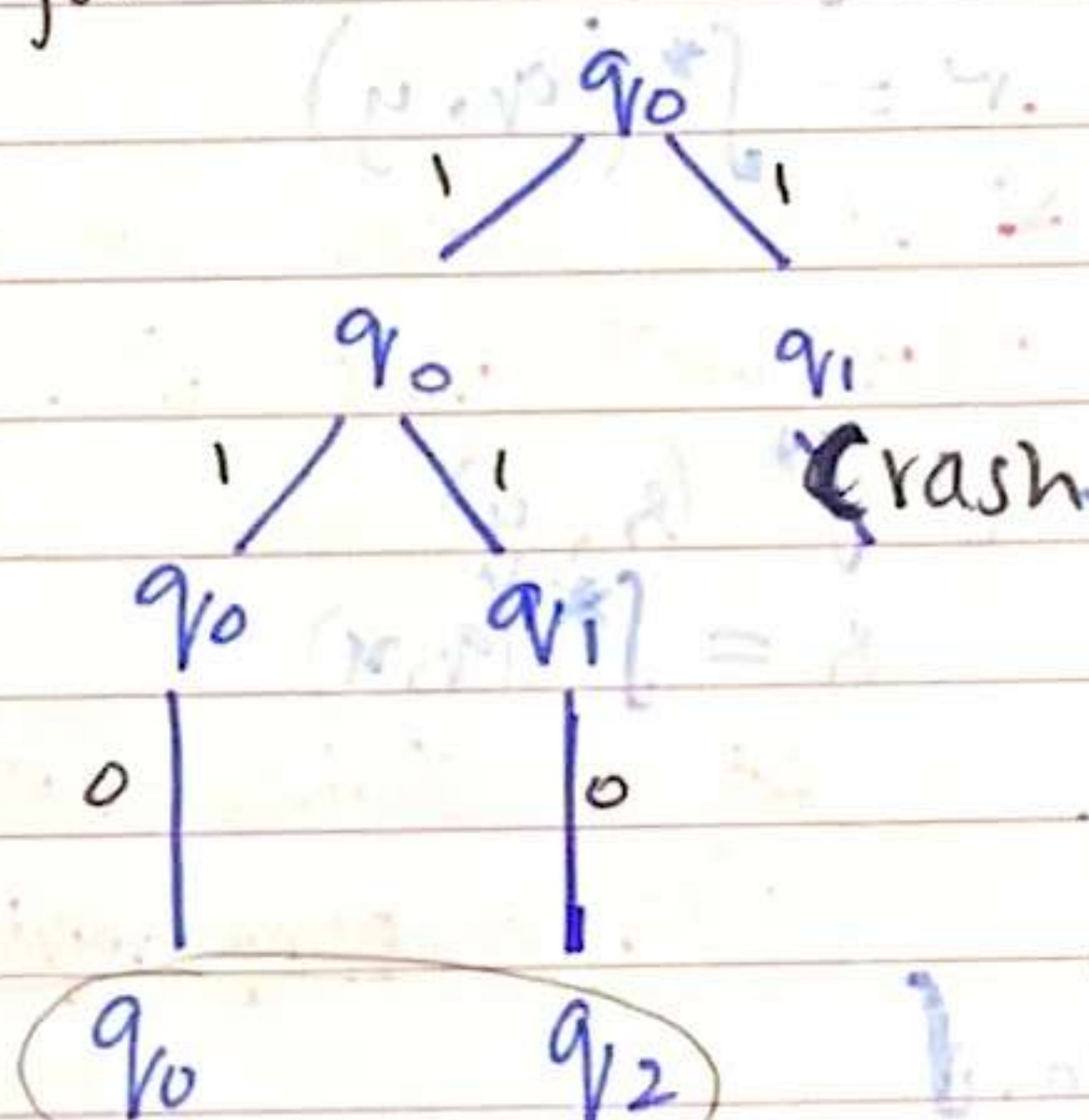
→ running for '110': $q_0 \xrightarrow{1} q_1 \xrightarrow{1} q_1 \xrightarrow{0} q_2$

'110' accepted since q_2 is final

NFA:

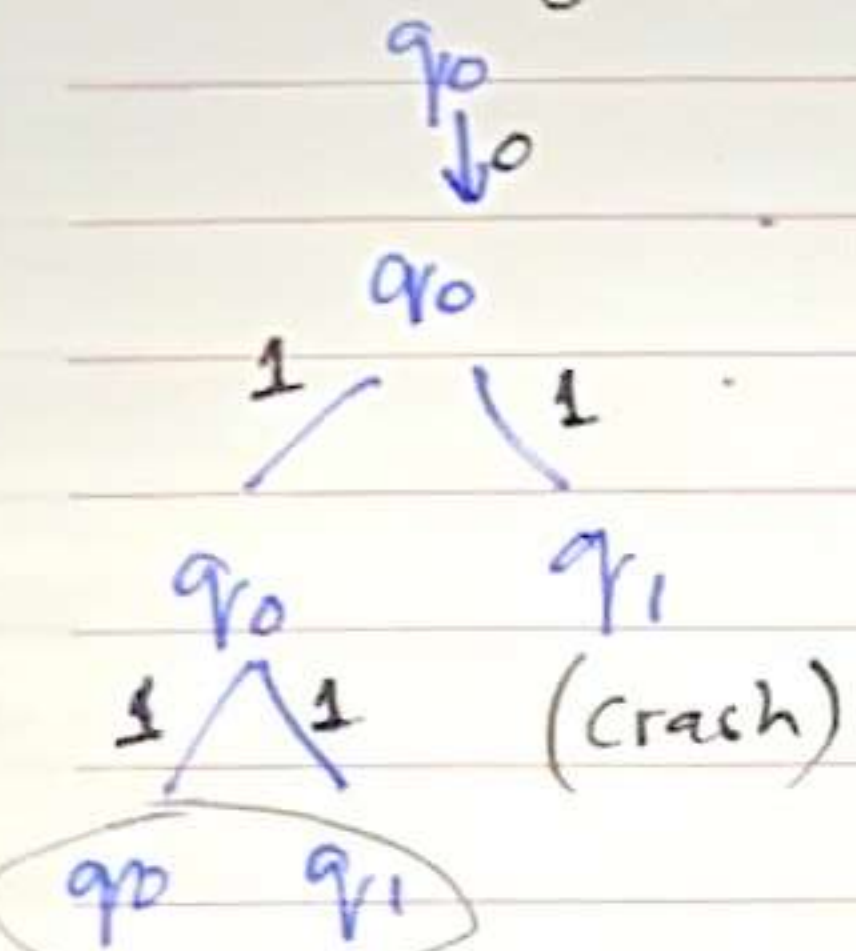


→ running for '110':



* Since the final state (i.e. q_2) is in the leaf nodes of NFA, ∴ string '110' is acceptable.

→ Running for '011'



→ since leaf nodes doesn't have 'q2' (final state) ∴ '011' is not acceptable.

⇒ Mathematical proving a string is part of a language:
Use extended func. & its property for NFA.

→ Properties for Extended Func.

DFA	NFA
① $\int^*(q, \lambda) \rightarrow q$	① $\int^*(q, \lambda) \rightarrow q$
② $\int^*(q, ya) \rightarrow \int(\int^*(q, y), a)$	② $\int^*(q, ya) \rightarrow \bigcup \int(r, a)$ $r = \int^*(q, y)$ means we'll get set of states instead of 1 state unlike in DFA ↓ set of states we will get after we'll input 'y'.
③ $\int^*(q, xy) \rightarrow \int^*(\int^*(q, x), y)$	③ $\int^*(q, xy) \rightarrow \bigcup \int^*(s, y)$ $s = \int^*(q, x)$ ↓ set of states we'll get after chopping right most element(y)

DFA	NFA
$Q, \Sigma, q_0, A_0, \delta$	$Q, \Sigma, q_0, A_0, \delta$

* Difference b/w DFA & NFA stuff lies only in extend func. $\int^*$.

$$\int(Q, a) \rightarrow Q \quad \int(Q, a) \rightarrow 2^Q$$

→ NFA on prev. page : $q_0 \xrightarrow{0} q_1 \xrightarrow{1} q_2$
⇒ Using Extended Func. to prove '110' is acceptable.

$\int^*(q_0, 110)$ (RM)
* chopping right most element(0)
→ Rule 2

$$\textcircled{1} = \bigcup \int(r_1, 0) \quad \bigcup \int(\int^*(q_0, 1), 0)$$

$$r_1 = \int^*(q_0, 1) \quad r_1$$

* Chopping RM '1' from r_1
 $\textcircled{2} = \bigcup \int(r_2, 1) \quad \bigcup \int(\int^*(q_0, \lambda), 1)$
 $r_2 = \int^*(q_0, \lambda)$

* Concatenating ' λ ' with 1
→ Rule 1

$r_2 = \int^*(q_0, \lambda \cdot 1)$
* Chopping RM 1 from r_2

$$\textcircled{3} = \bigcup \int(r_3, 1)$$

$$r_3 = \int^*(q_0, \lambda)$$

* Rule 1: $r_3 = \int^*(q_0, \lambda) = q_0$
∴ $r_3 = \{q_0\}$

• BACK-TRACK

→ * Putting this value of $r_3 = \{q_0\}$ into $\textcircled{3}$

$$\bigcup \int(q_0, 1)$$

* input = 1 on q_0 gives two next states: $\{q_0, q_1\}$

* Putting this in $\textcircled{2}$

$$= \bigcup \int(r_2, 1)$$

$$= \bigcup \int(\{q_0, q_1\}, 1)$$

$$= \int(q_0, 1) \cup \int(q_1, 1)$$

$$= \{q_0, q_1\} \cup \emptyset$$

$$= \{q_0, q_1\}$$

* Putting this in $\textcircled{1}$

$$= \bigcup \int(r_1, 0)$$

$$= \bigcup \int(\{q_0, q_1\}, 0)$$

$$= \int(q_0, 0) \cup \int(q_1, 0)$$

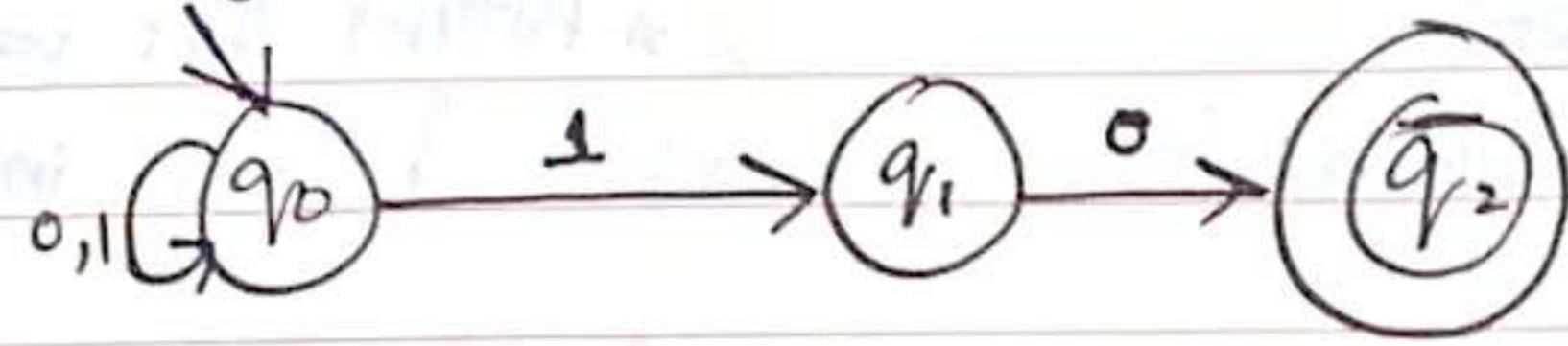
$$= \{q_0\} \cup \{q_2\}$$

$$\therefore \text{answer} = \{q_0, q_2\}$$

∵ since q_2 is final state is in answer;
∴ 110 is acceptable

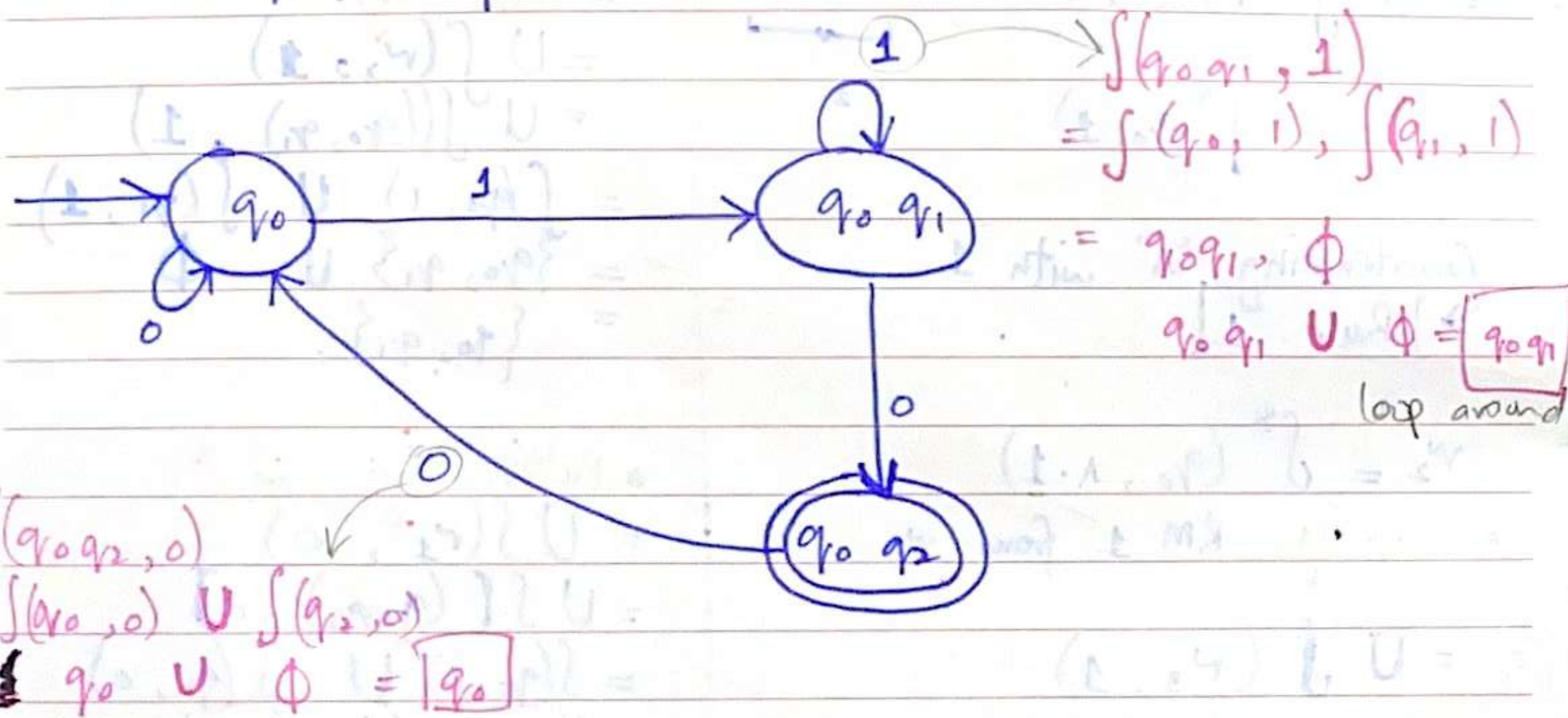
GIST: After NFA's table, we'll start giving input 0 & 1 to initial state, and then jo answer main state(s) aise gen unpar we'll give 0 & 1 input & phir move on & on....

□ Making DFA from NFA:

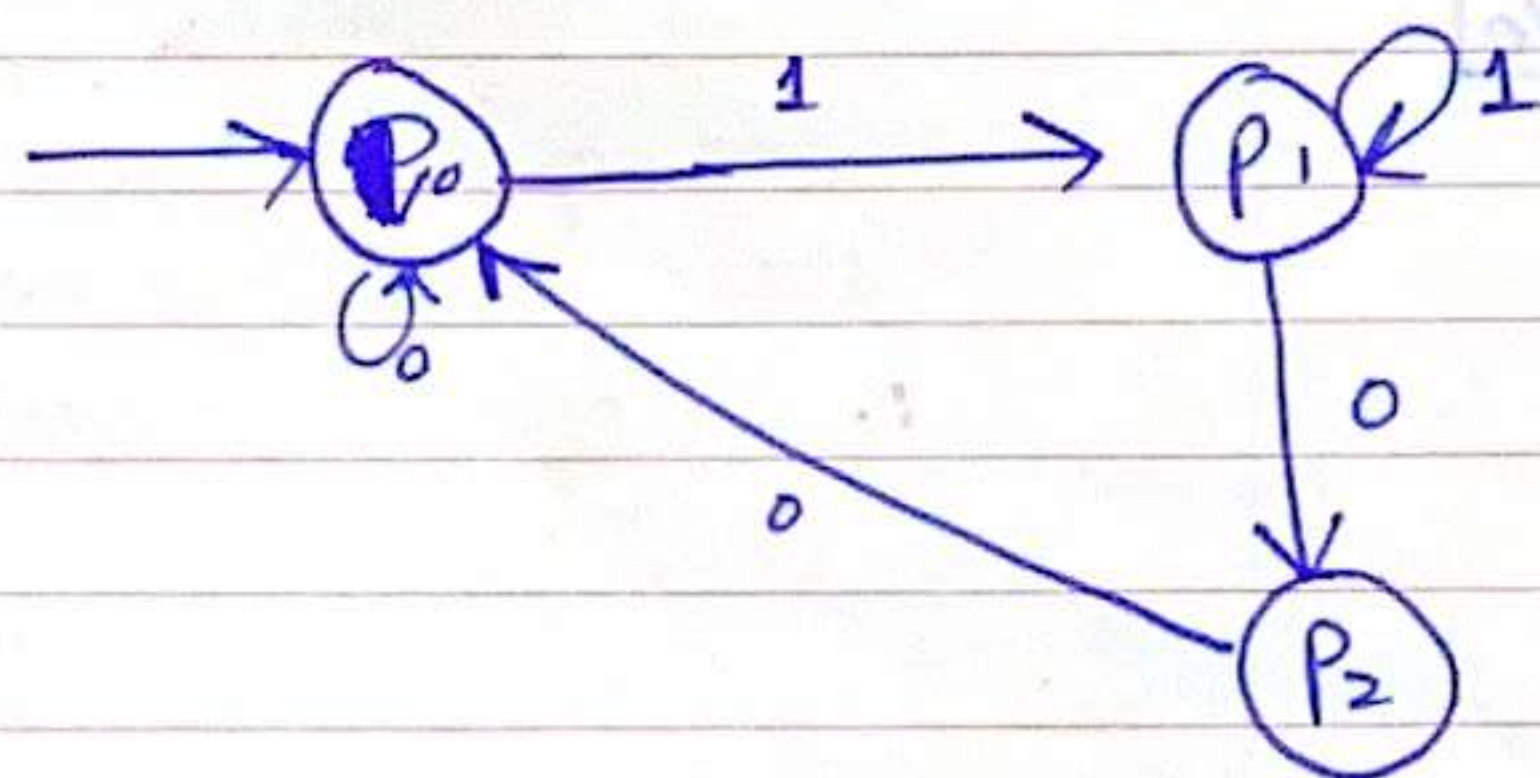


	0	1
q ₀	q ₀	{q ₀ , q ₁ }
q ₁	q ₂	∅
q ₂	∅	∅

next no state on this input



into new variables



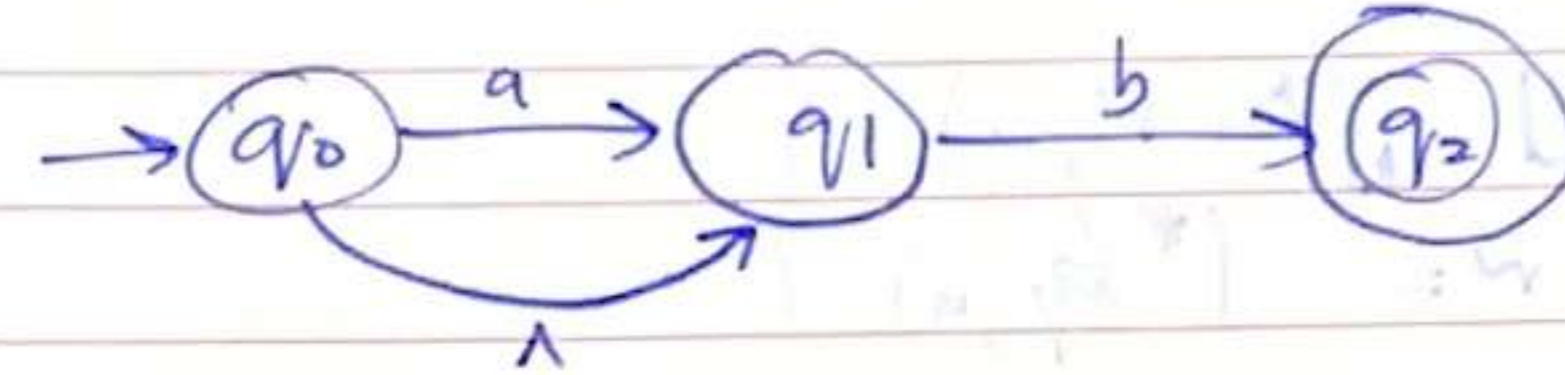
* Now it is DFA since each state have exactly one arc for each input (0 & 1)

day / date:

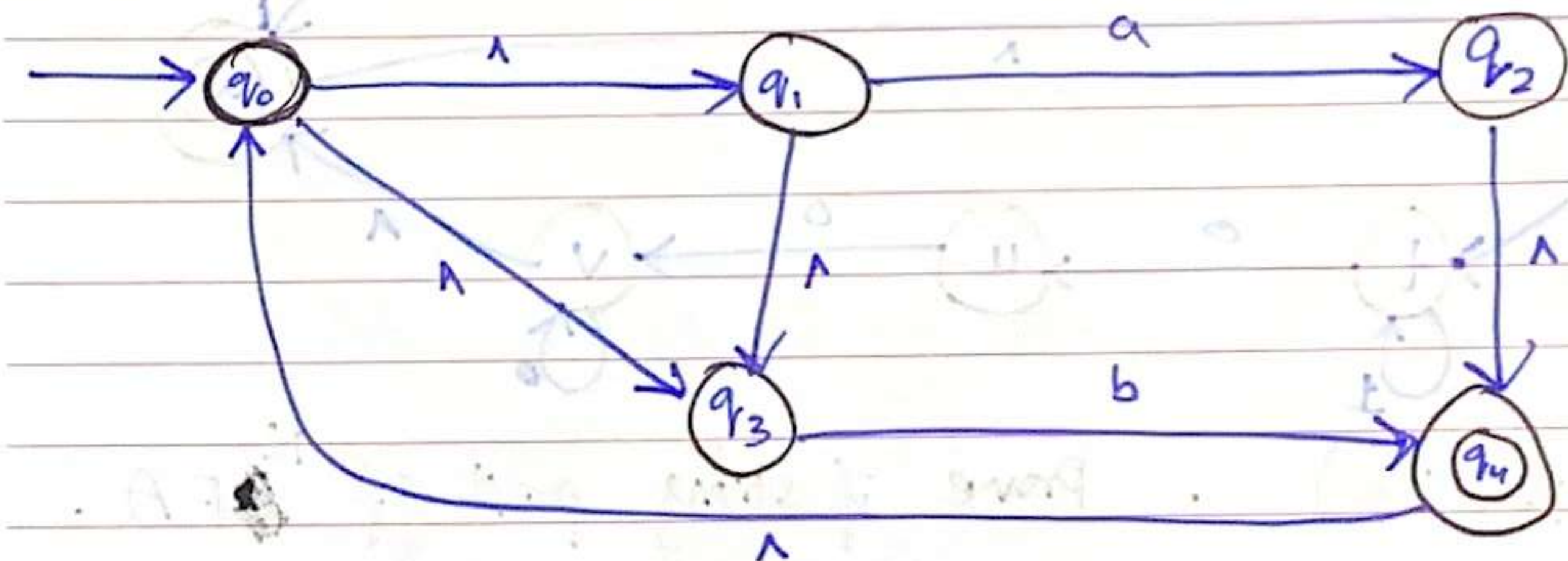
LECTURE - 8 Thursday 11/9/25

□ NFA NULL: NFA + Λ transition

Reg. Expression: $(ab + a)$



⇒ Null Closure: How many states can I jump from a specific state without consuming any input (by js consuming Λ).



jiska null closure berhan that state will always be included in Λ -closure

- $\Lambda\{q_0\} \rightarrow \{q_0, q_1, q_3\}$
- $\Lambda\{q_1\} \rightarrow \{q_1, q_3\}$
- $\Lambda\{q_2\} \rightarrow \{q_2, q_4, q_0, q_1, q_3\}$
- $\Lambda\{q_3\} \rightarrow \{q_3\}$
- $\Lambda\{q_4\} \rightarrow \{q_4, q_0, q_1, q_3\}$

Properties of Extended Func. [NFA] ^{NOT SIMPLE NFA} NULL

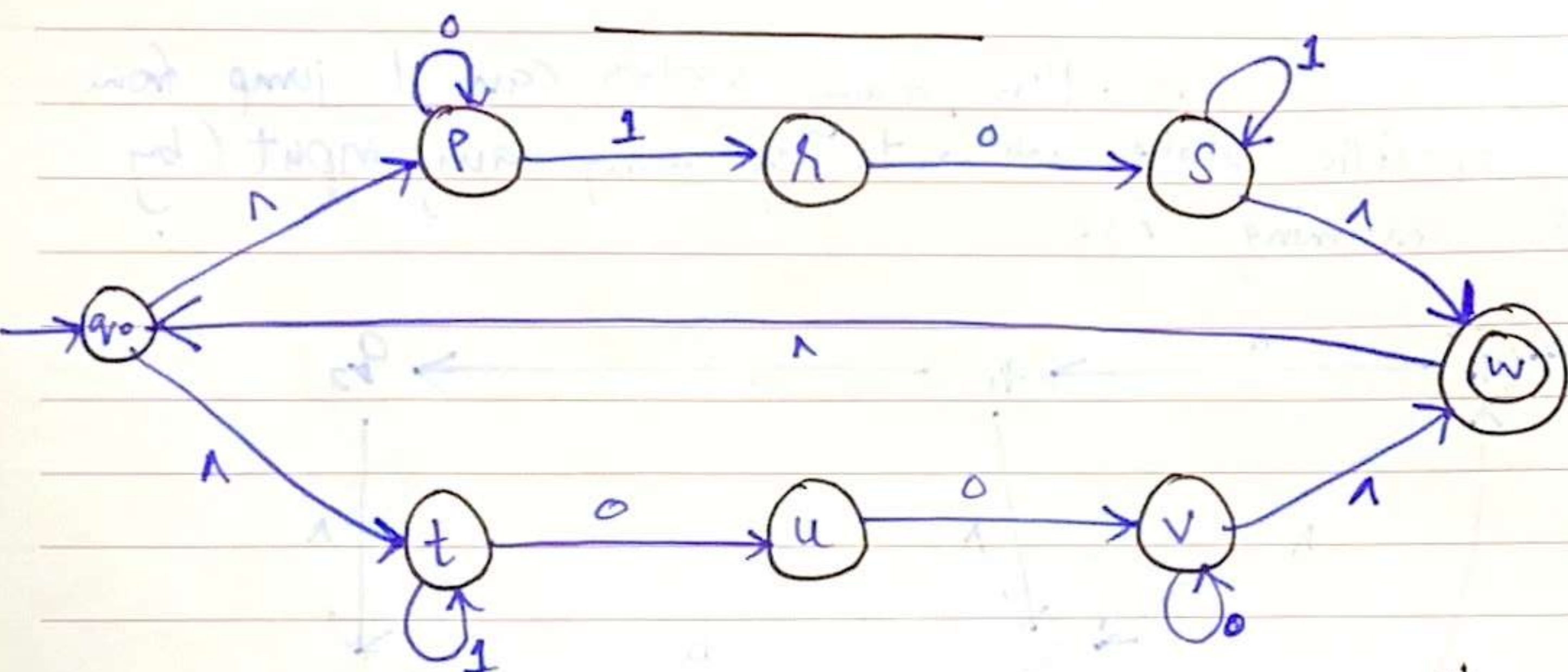
day / date:

Rule ①

$$① \int^*(q, \Lambda) \rightarrow \Lambda \{q\} \quad (\text{null closure of } q).$$

$$② \int^*(q, ya) \Rightarrow \Lambda \left\{ \bigcup_{r = \int^*(q, y)} \int(r, a) \right\}$$

$$③ \int^*(q, uy) \Rightarrow \Lambda \left\{ \bigcup_{w = \int^*(q, u)} \int^*(w, y) \right\}$$



Q: $\int^*(q_0, 100)$. Prove if string part of NFA.

$$= \Lambda \left\{ \bigcup_{r_1 = \int^*(q_0, 10)} \int(r_1, 0) \right\} \rightarrow ①$$

$$= \int^*(q_0, 10) = \Lambda \left\{ \bigcup_{r_2 = \int^*(q_0, 1)} \int(r_2, 0) \right\} \rightarrow ②$$

$$\int^*(q_0, 101) = \Lambda \left\{ \bigcup_{r_3 = \int^*(q_0, 10)} \int(r_3, 1) \right\} \rightarrow ③$$

$$\int^*(q_0, \Lambda) = \Lambda \{q_0\}$$

$$q_0 \rightarrow \text{null closure} = \{q_0, p, t\}$$

* Put this in eq ③

$$= \Lambda \left\{ \bigcup_{r_3 = \{q_0, p, t\}} \int(r_3, 1) \right\} \rightarrow \Lambda \left\{ \int(q_0, 1) \cup \int(p, 1) \cup \int(t, 1) \right\}$$

$$= \Lambda \{\phi \cup r \cup t\} \rightarrow \Lambda \{r, t\}$$

$$\Lambda \{r, t\} = \Lambda \{r\} \cup \Lambda \{t\}$$

(null closure of r) $\cup$ (null closure of t)

$$= \{r, t\}$$

$$r_2 = \{r, t\}$$

* Put this in eq ② in place of r_2 .

$$②: \int^*(q_0, 1) = \Lambda \left\{ \bigcup_{r_2 = \{r, t\}} \int(r_2, 0) \right\}$$

$$= \Lambda \left\{ \int(r, 0) \cup \int(t, 0) \right\} = \Lambda \{s, u\}$$

$$\Lambda \{s, u\} = \{s, w, q_0, p, t, u\}$$

* Put this in eq ① in place of r_1 .

$$①: r_1 = \int^*(q_0, 10) \text{ at } \int(r_1, 0) = \{s, w, q_0, p, t, u\}$$

$$= \Lambda \left\{ \int(s, 0) \cup \int(w, 0) \cup \int(q_0, 0) \cup \int(p, 0) \cup \int(t, 0) \cup \int(u, 0) \right\}$$

$$= \Lambda \{\phi \cup \phi \cup \phi \cup p \cup u \cup v\}$$

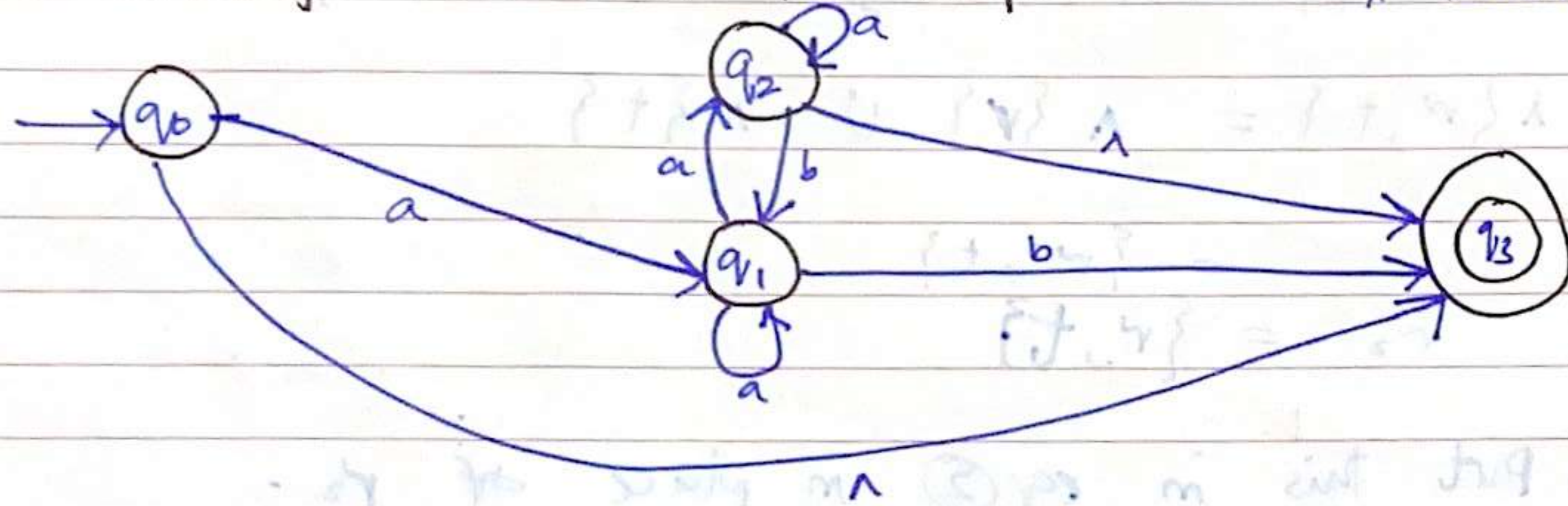
day / date:

$$= \Lambda \{P, u, v\} \quad (\text{Null closure of } P, u, v)$$

$$= \{P, u, v, w, q_0, t\}$$

since 'w' i.e. final state is in the answer $\therefore$ '100' is acceptable string.

□ Converting NFA Null into Simple NFA: Remove all transitions



	$\delta(q, a)$	$\delta(q, b)$	$\Lambda \{q\}$	$\delta^*(q, a)$	$\delta^*(q, b)$
q_0	$\{q_1\}$	$\emptyset$	$\{q_0, q_3\}$	q_1 meow	$\emptyset$
q_1	$\{q_2, q_1\}$	$\{q_3\}$	$\{q_1\}$	$\{q_1, q_2, q_3\}$	$\{q_3\}$
q_2	$\{q_2\}$	$\{q_1\}$	$\{q_2, q_3\}$	$\{q_2, q_3\}$	$\{q_1\}$
q_3	$\emptyset$ (wash)	$\emptyset$	$\{q_3\}$	$\emptyset$	$\emptyset$

$$\delta^*(q_0, a) = \delta^*(q_0, \Lambda \cdot a) = \Lambda \left\{ \bigcup_{r \in \delta^*(q_0, \Lambda)} \delta(r, a) \right\}$$

$$\delta^*(q_0, \Lambda) = \{q_0, q_3\}$$

$$= \Lambda \left\{ \delta(q_0, a) \cup \delta(q_3, a) \right\}$$

$$= q_1 \cup \emptyset$$

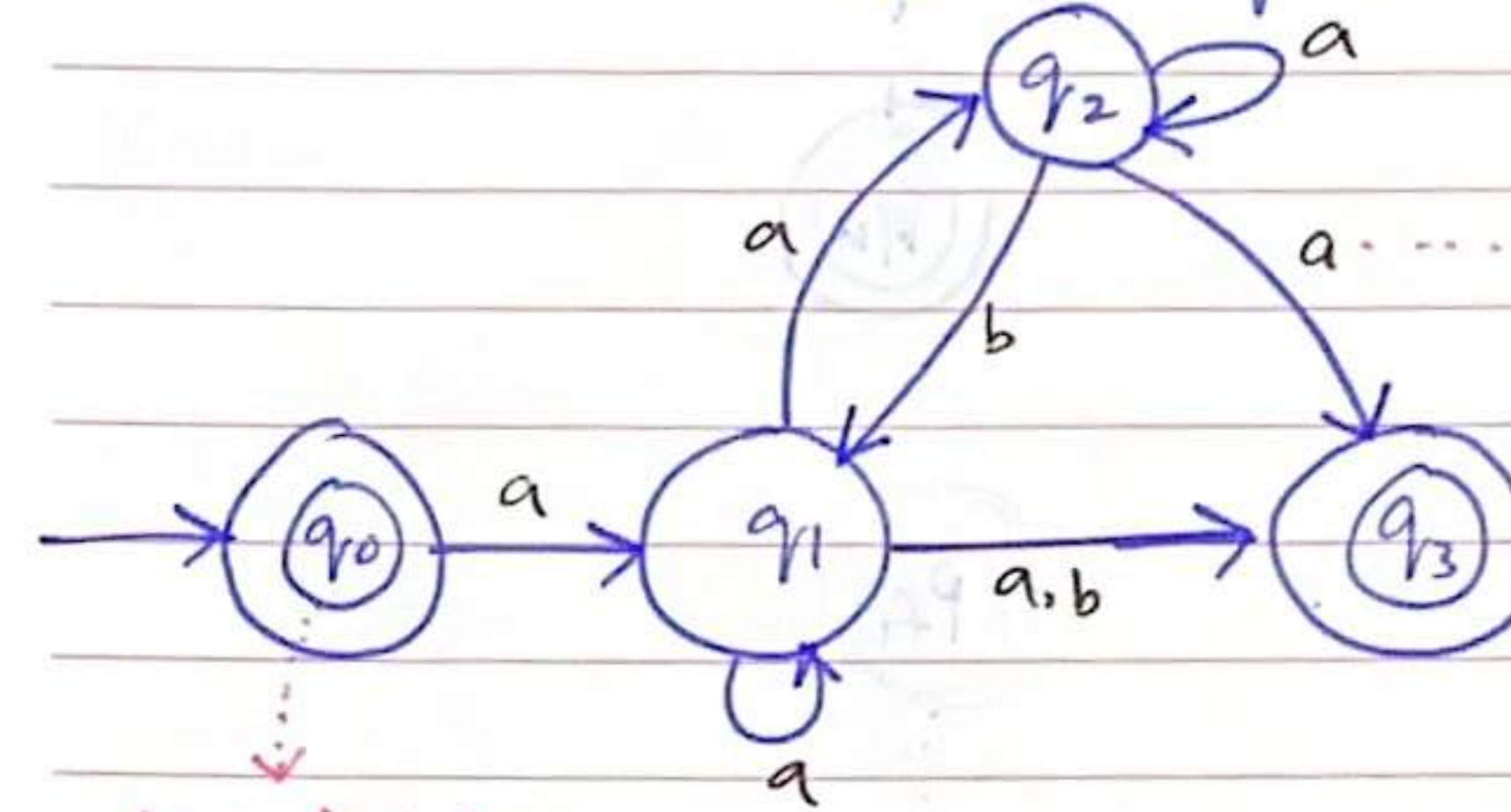
$$\text{meow} = \Lambda \{q_1\} = \{q_1\}$$

if original final state in NFA null is in any set of null closure of states; then that state will become final as well.

only $\delta^*(q, a)$ and $\delta^*(q, b)$ columns used to make NFA.

day / date:

→ Now making simple NFA using table:



q_0 is final too since Λ is acceptable on initial stage (q_0).

came because of the loop around 'a' on q_2 .
• Agar q_2 par kuch nai input hoga (which is null), so we have two possibility:

- $\delta(q_2, a) = q_2$
- $\delta(q_2, a) = q_3$

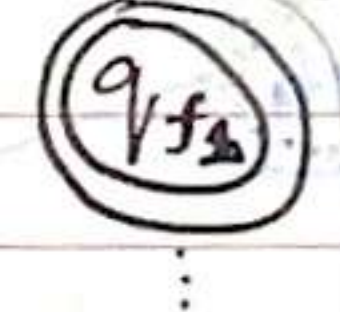
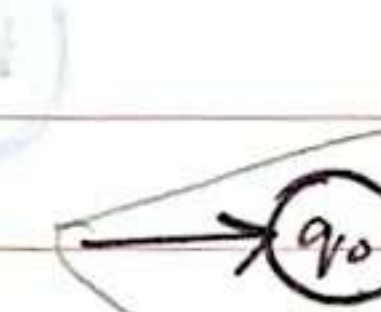
□ Kleene's Theorem.

LECTURE-9

Tuesday
16/9/25

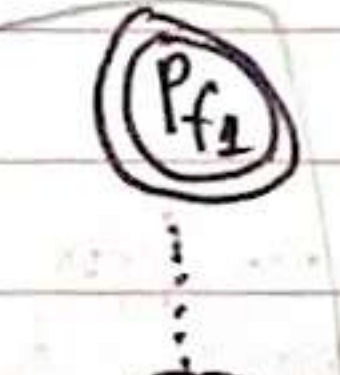
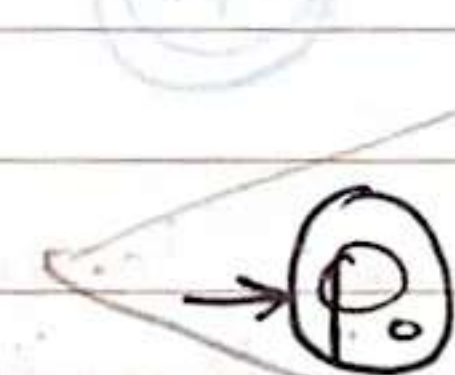
Reg Expression $\rightarrow$ NFA $\rightarrow$ NFA $\rightarrow$ DFA

L_1



all states of L_1 $(Q_1, \Sigma, q_0, A_1, \delta_1)$
initial state q_0
final states q_1

L_2



$(Q_2, \Sigma, p_0, A_2, \delta_2)$

$$L_3 = (Q_u, \Sigma, q_u, A_u, \delta_u)$$

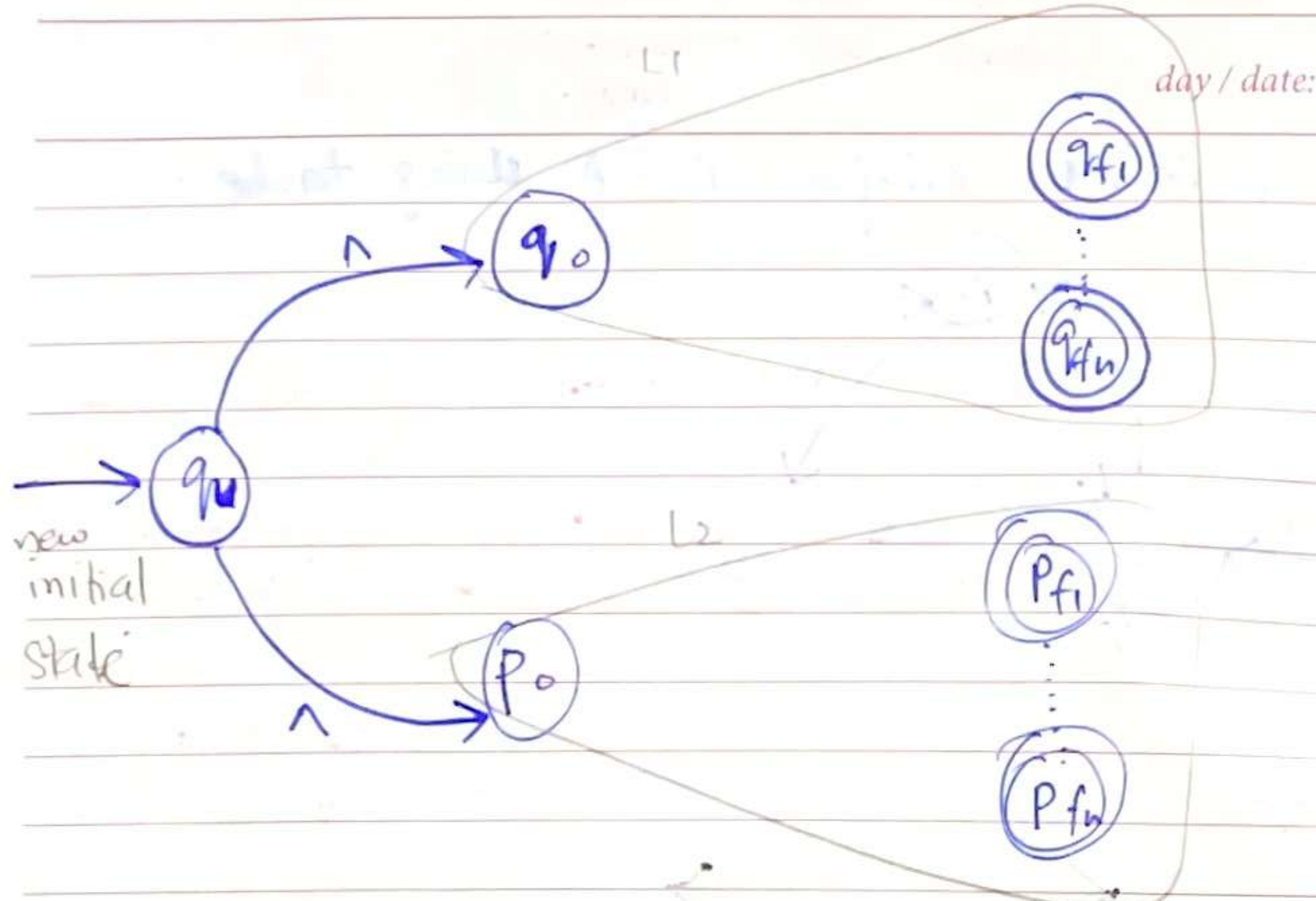
$$q_u = q_0$$

$$Q_u = Q_1 \cup Q_2 \cup q_u \quad (\text{all states of } L_1 \text{ \& } L_2 \text{ with } q_u)$$

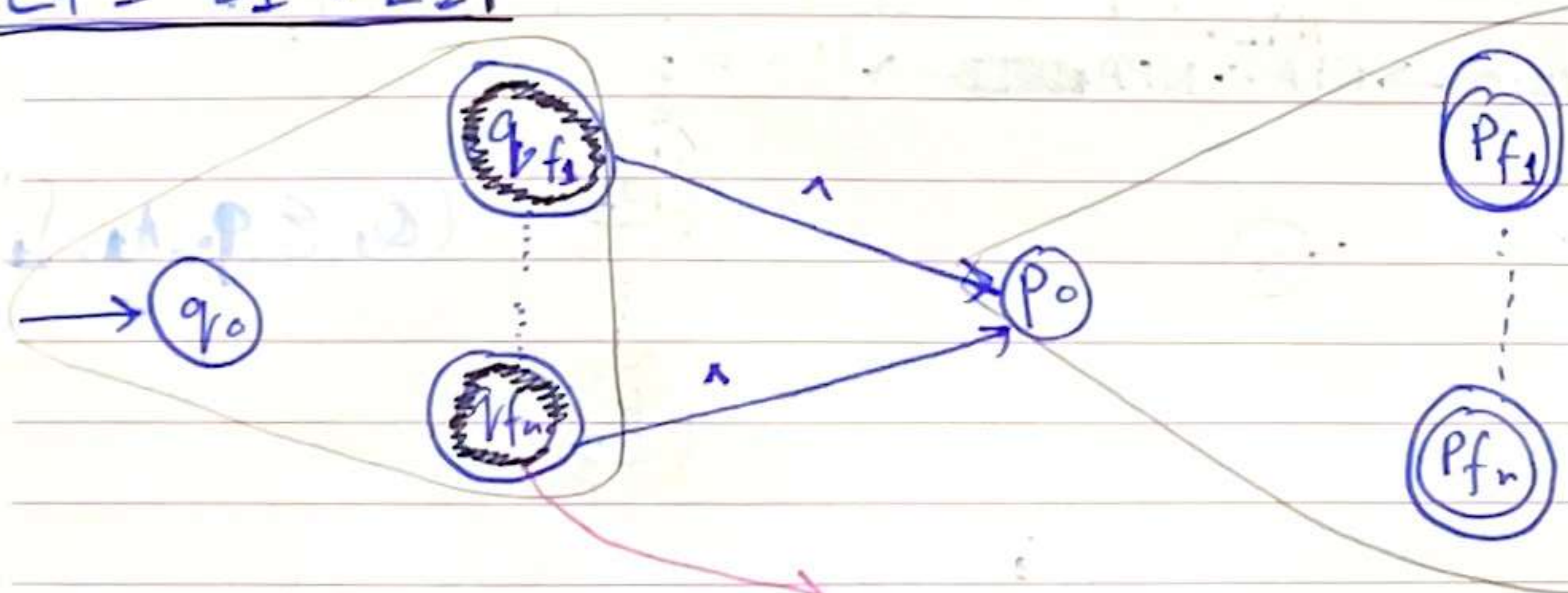
$$A_u = A_1 \cup A_2 \quad (\text{all final states of } L_1 \text{ \& } L_2)$$

$$\delta_u = \delta_1 \cup \delta_2 \cup \delta_u(q_u, \Lambda) \rightarrow \{q_0, p_0\}$$

first one state connecting both diagrams



$$L_4 = L_1 \cdot L_2$$



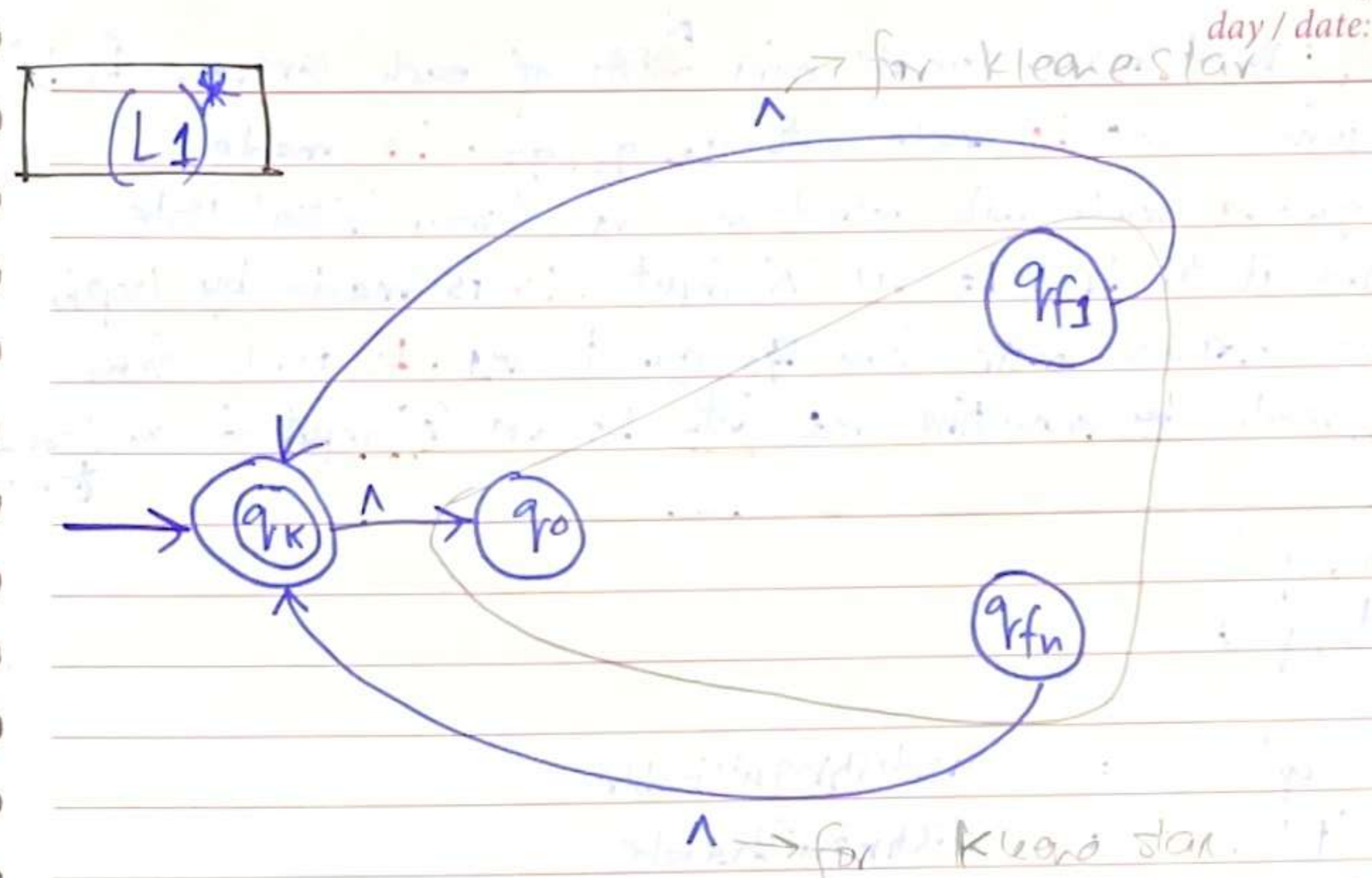
$(Q_c, \Sigma, q_c, A_c, f_c)$ $\rightarrow q_{f1} \dots q_{fn}$ are no longer final states, they're intermediate states.

$$Q_c = Q_1 \cup Q_2$$

$$q_c = q_0$$

$$A_c = A_2 \text{ (only } L_2 \text{ final state)}$$

$$f_c = f_1 \cup f_2 \cup f_c (q_{fi}, \lambda) \rightarrow p_0$$



★ Dummy (initial + final) state i.e. q_k is required since if we have a self-loop on initial, it won't work without dummy state.

$\Rightarrow$ NFA NULL FROM REG. EXPRESSION.

$$Q: (11 + 0)^* 00$$

$$L_1 = 11$$

$$L_2 = 0$$

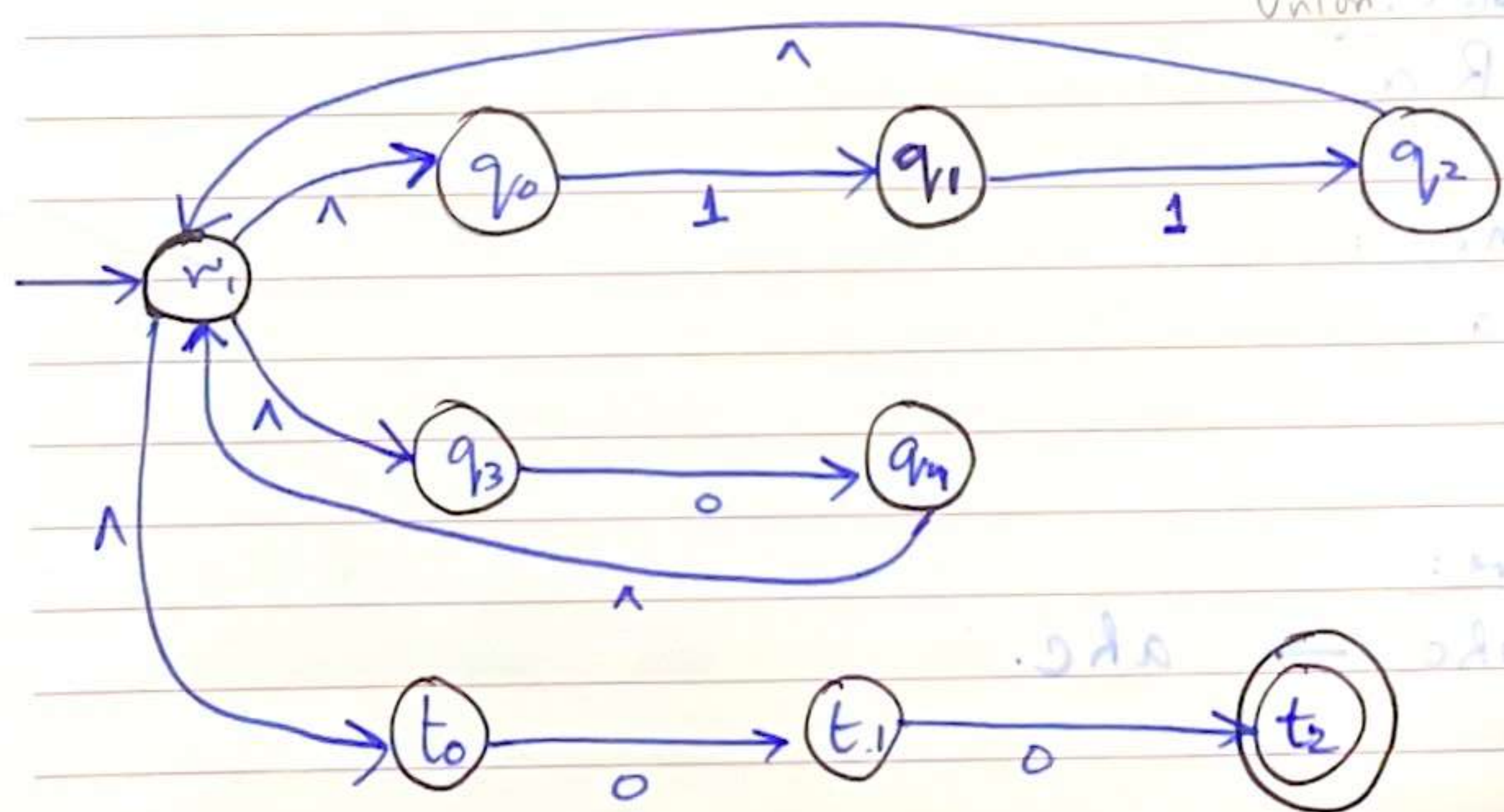
$$L_3 = (11 \cup 0)^*$$

$$L_4 = (11 \cup 0)^*$$

$$L_5 = 00$$

$$L_6 = (11 \cup 0)^* \cdot 00$$

Union Concatenation



METHOD: We'll make separate mini NFAs of each language (L_1 - L_6) and then join them. L_1 made with q_0, q_1, q_2 . L_2 made with q_3, q_4 . L_3 made with introducing r_1 dummy initial state and connecting it to L_1 & L_2 via 'a' input. L_4 is made by looping around from arrows coming from q_2, q_4 to r_1 . L_5 made from t_1, t_2 . L_6 made by connecting r_1 with L_5 via 'a' input for concatenation.

$A = \{s, q\}$ A is set of final states
 $Z = \{t, p\}$ Z is set of intermediate states.

's' and 'q' : Indistinguishable
 's' and 't' : distinguishable
 't' and 'p' : Indistinguishable

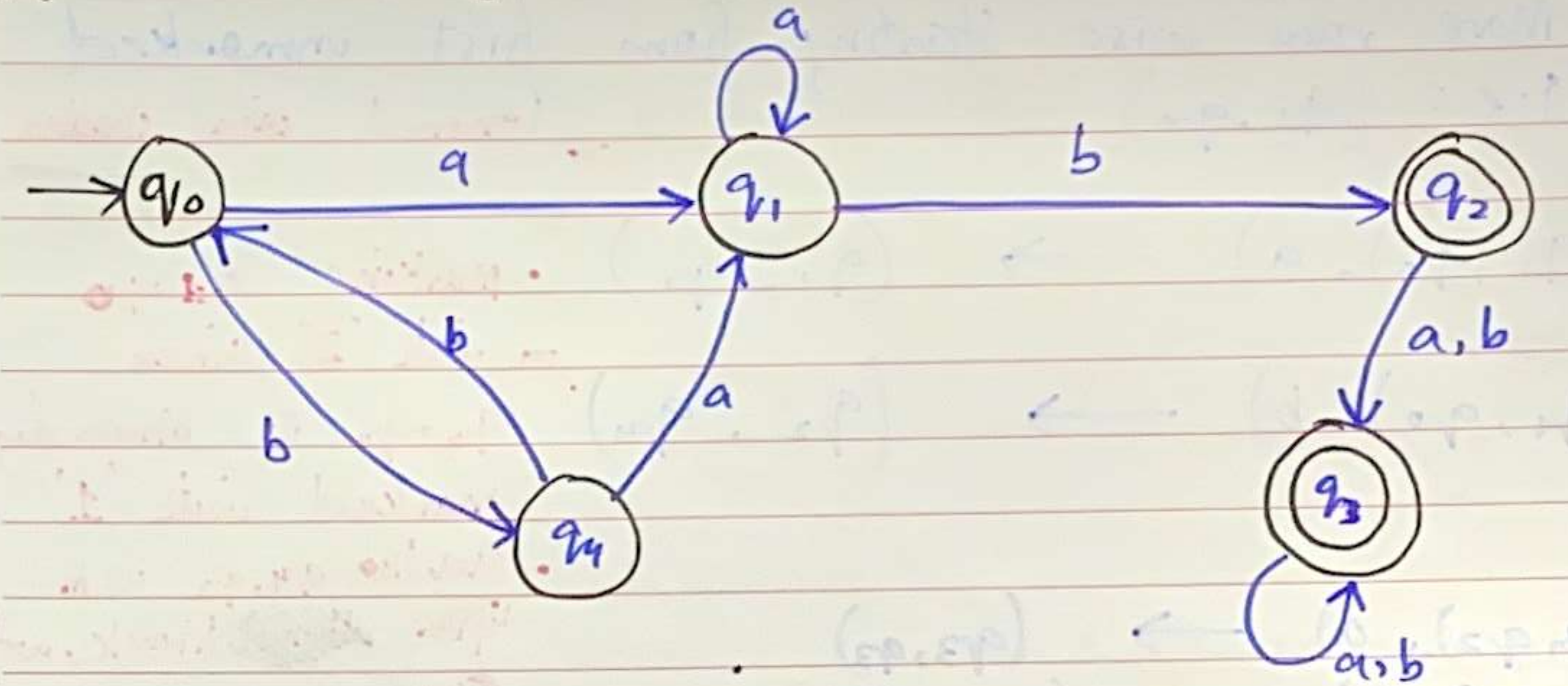
NOTE: We can only merge "Indistinguishable" states.
 (e.g: final states or intermediate can merge with final or intermediate states respectively).
 • Zarori nahi that we can merge all indistinguishable states
 $\Rightarrow$ **Equivalence Relation**: All 3 properties shld hold.
 — Indistinguishable is equivalence relation

① Reflexive:
 $a R a$ (Bsf of ourselves)

② Symmetric:
 $a R b, b R a$ (a is sibling of b
 b is sibling of a)

③ Transitive:
 $a R b, b R c \rightarrow a R c$. a is sibling of b, b sibling of c;
 so a sibling of c

Minimising DFA



	q_0	q_1	q_2	q_3	q_4
q_0					
q_1	2				
q_2	1	1			
q_3	1	1			
q_4		2	1	1	
	q_0	q_1	q_2	q_3	q_4

→ relation b/w q_2 & q_4
 or q_1 & q_3
 (order doesn't matter)

① Diagonals are ignored since if it is Indistinguishable relation; then it is equivalence relation... reflexive property holds (i.e: q_1, q_1 ; q_3, q_3)

② Ignore upper triangular half since it is symmetric to lower half of triangle.
 the ~~cells~~ ^{cells} ~~may~~ ^{may} final states included here are distinguishable

③ Mark distinguishable states (that cannot be merged) with '1'

④ Check behaviour of unmarked cells. They're Indistinguishable but it's not necessary they all could be merged. So we'll check behaviour in ⑤.

→ all where one state is final & one is not

* If we checked an unmarked cell with input 'a' and the result is ignored; only then we'll take input 'b' on same cell. If on input 'a' the cell gets marked by 2; we wait check for input 'b'.

⑤ Move row wise starting from first unmarked
i.e.: (q_1, q_0)

$\delta((q_1, q_0), a) \rightarrow (q_1, q_1)$
 $\delta((q_1, q_0), b) \rightarrow (q_2, q_4)$

• marking q_1, q_0 with '2' since q_4, q_2 is already marked with '1'
 • Also q_4, q_2 is in upper triangle, we'll still check q_2, q_4 if it is marked with 1

• $\delta((q_3, q_2), a) \rightarrow (q_3, q_3)$
 $\delta((q_3, q_2), b) \rightarrow (q_3, q_3)$

ignored since diagonal shaded

• $\delta((q_4, q_0), a) \rightarrow (q_1, q_1)$
 $\delta((q_4, q_0), b) \rightarrow (q_0, q_4)$

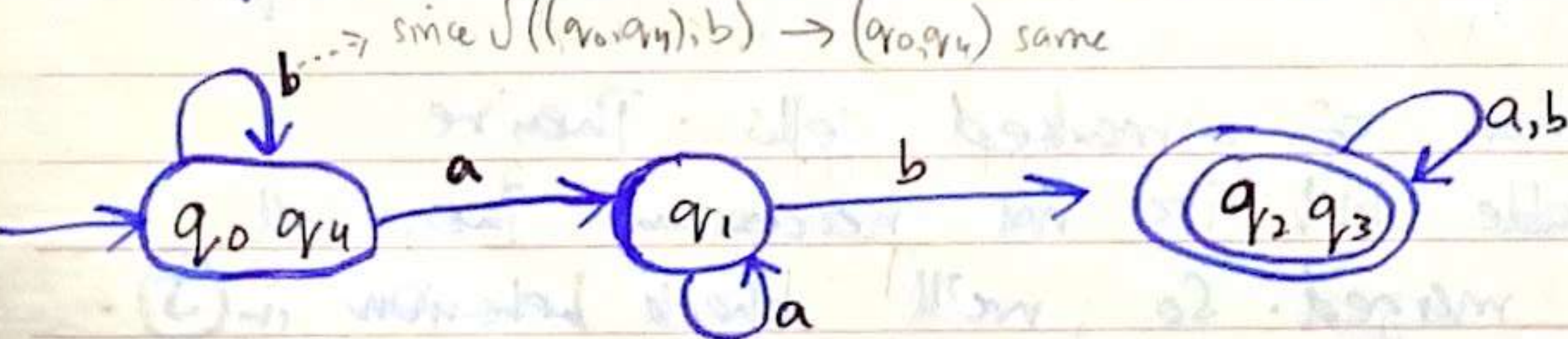
ignored since unmarked

• $\delta((q_4, q_1), a) \rightarrow (q_1, q_1)$
 $\delta((q_4, q_1), b) \rightarrow (q_0, q_2)$

marking (q_4, q_1) with 2 since (q_0, q_2) is marked with 1

⑥ Now check unmarked states again with same logic in ⑤.
 Mark any unmarked box with '3' only if the output state (after input 'a' or 'b') has '1' or '2' marked on it.

⑦ If there is a '3' after step ⑥, check unmarked cells again. Other wise move column wise from left and merge states to form DFA as the unmarked states now are indistinguishable.



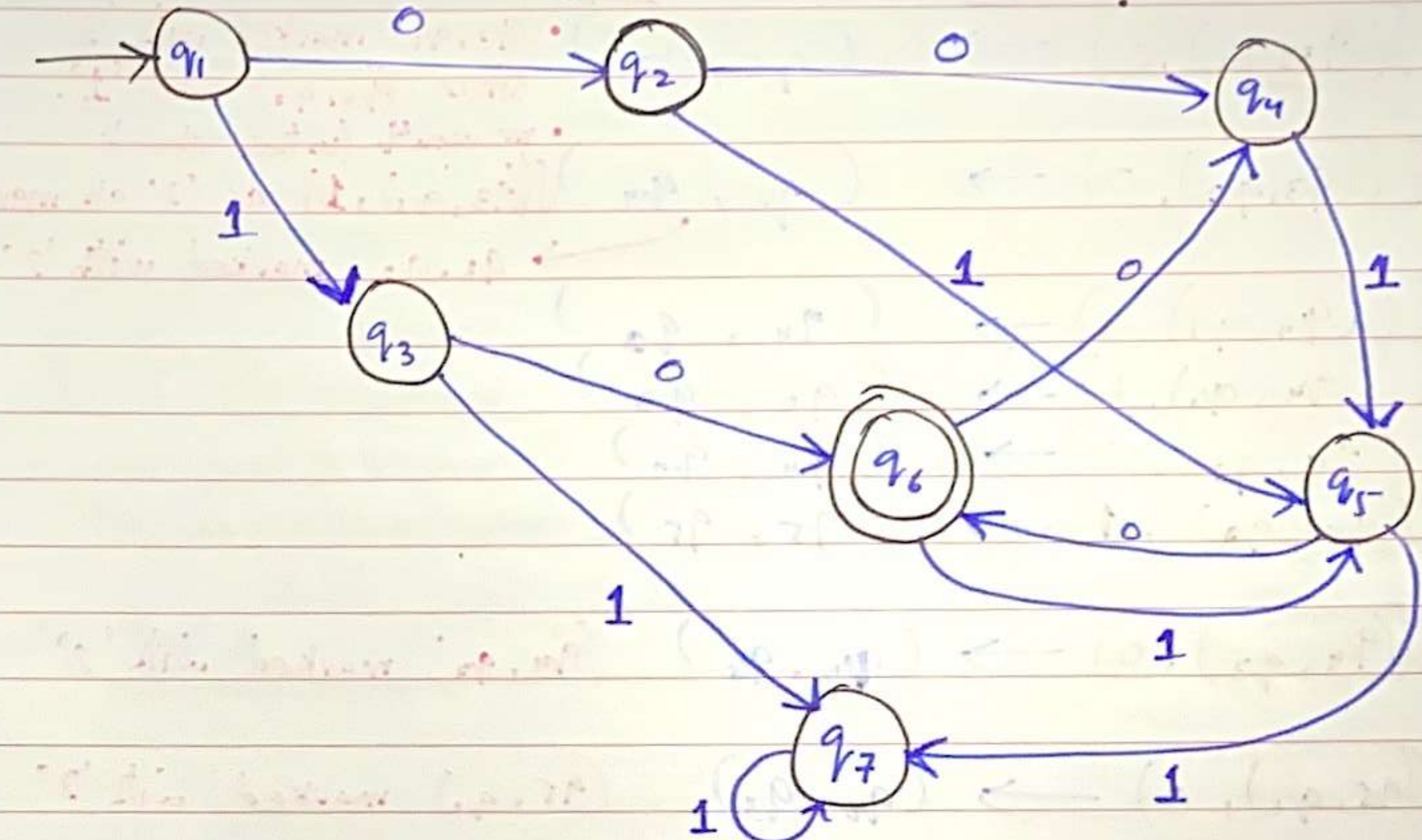
* Since DFA so give both input a & b on each state to find every possible combination

day / date:

LECTURE - 10

Thursday
18/9/25

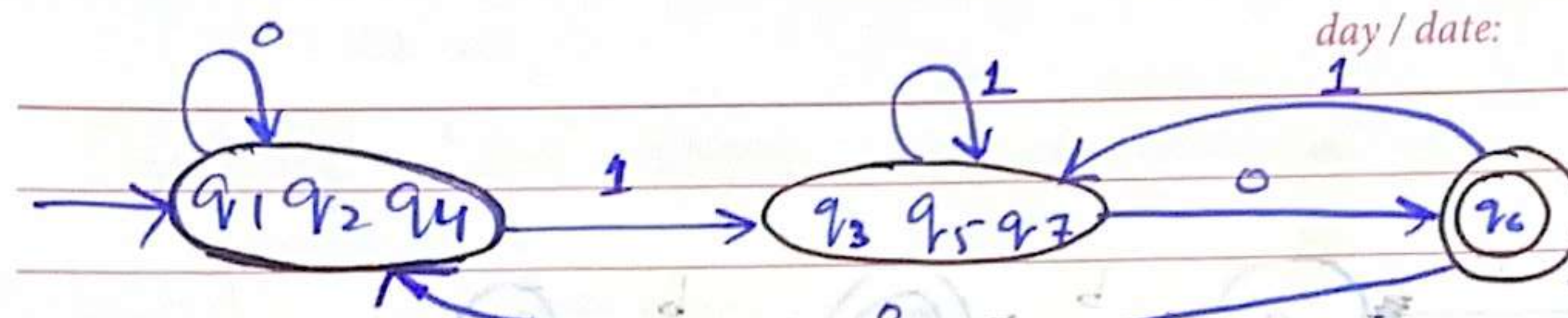
□ Minimising DFA (another example)



q_1	2	2	2	2	2	2	2
q_2		2	2	2	2	2	2
q_3	2	2	2	2	2	2	2
q_4			2	2	2	2	2
q_5	2	2		2	2	2	2
q_6	1	1	1	1	1	1	1
q_7	2	2		2		1	1
	q_1	q_2	q_3	q_4	q_5	q_6	q_7

Step 2:

- $\int ((q_2, q_1), 0) \rightarrow (q_4, q_2)$ ignored } unmarked
 $\int ((q_2, q_1), 1) \rightarrow (q_5, q_3)$ ignored } unmarked
- $\int ((q_3, q_1), 0) \rightarrow (q_6, q_2)$ (q_3, q_1) marked with '2' since (q_6, q_2) had '1'. we won't further check
- $\int ((q_3, q_2), 0) \rightarrow (q_6, q_4)$ (q_3, q_2) marked with '2' as '2' already marked
- $\int ((q_4, q_1), 0) \rightarrow (q_4, q_2)$ ignored } unmarked
 $\int ((q_4, q_1), 1) \rightarrow (q_5, q_3)$ ignored } unmarked
- $\int ((q_4, q_2), 0) \rightarrow (q_4, q_4)$ ignored } upper triangle
 $\int ((q_4, q_2), 1) \rightarrow (q_5, q_5)$ ignored } shaded
- $\int ((q_4, q_3), 0) \rightarrow (q_4, q_6)$ (q_4, q_3) marked with '2'
- $\int ((q_5, q_1), 0) \rightarrow (q_6, q_3)$ (q_5, q_1) marked with '2'
- $\int ((q_5, q_2), 0) \rightarrow (q_6, q_4)$ (q_5, q_1) marked with '2'
- $\int ((q_5, q_3), 0) \rightarrow (q_6, q_6)$ ignored } unmarked
 $\int ((q_5, q_3), 1) \rightarrow (q_7, q_7)$ ignored } unmarked
- $\int ((q_5, q_4), 0) \rightarrow (q_6, q_4)$ (q_5, q_4) marked with '2'
- $\int ((q_7, q_1), 0) \rightarrow (q_6, q_2)$ (q_7, q_1) marked with '2'
- $\int ((q_7, q_2), 0) \rightarrow (q_6, q_4)$ (q_7, q_2) marked with '2'
- $\int ((q_7, q_3), 0) \rightarrow (q_6, q_6)$ ignored } upper triangle
 $\int ((q_7, q_3), 1) \rightarrow (q_7, q_4)$ ignored } shaded
- $\int ((q_7, q_4), 0) \rightarrow (q_6, q_4)$ (q_7, q_4) marked with '2'
- $\int ((q_7, q_5), 0) \rightarrow (q_6, q_6)$ ignored } diagonal
 $\int ((q_7, q_5), 1) \rightarrow (q_7, q_7)$ ignored } diagonal



* To check if this merge DFA is correct; tally any state from 1 bubble with single input & the next state should be inside the destination bubble.

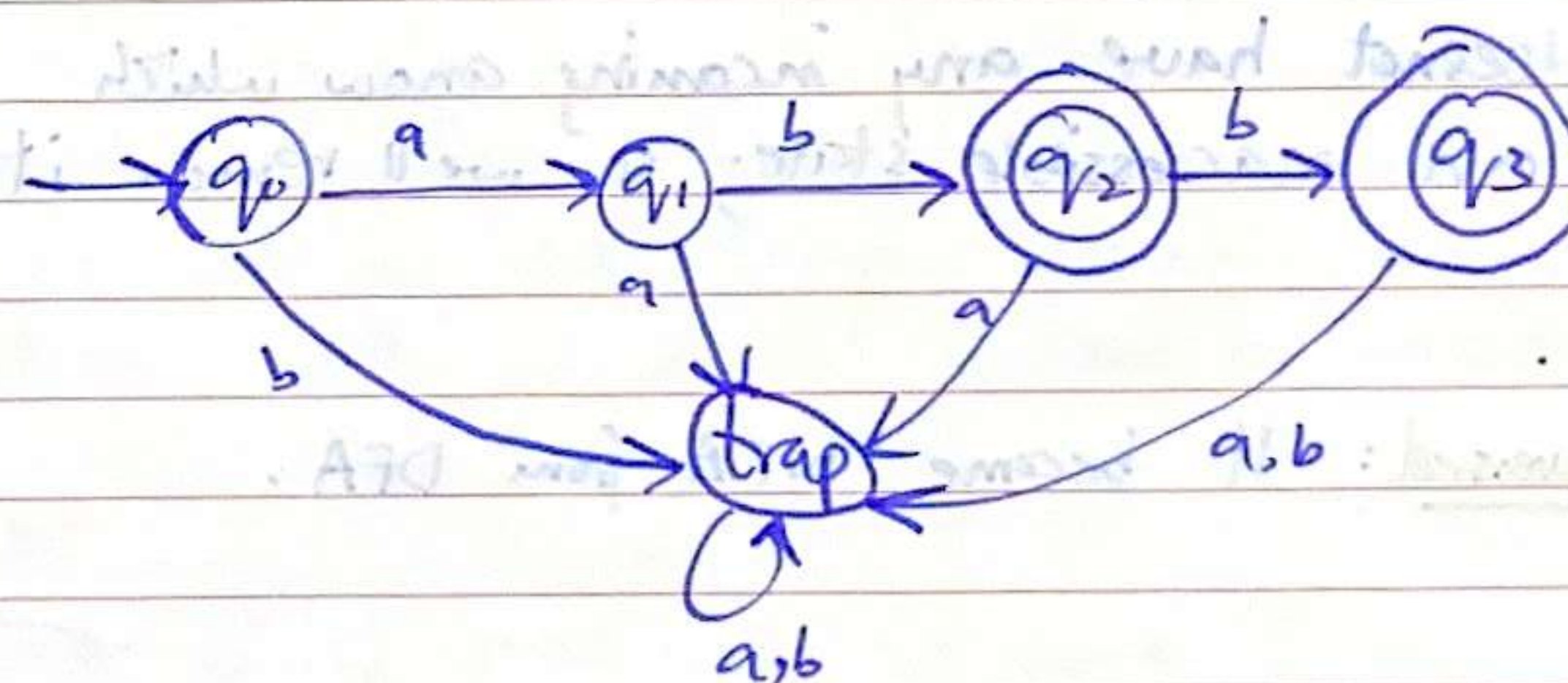
$(q_1, 1) \rightarrow q_3$
 $(q_2, 1) \rightarrow q_5$
 $(q_4, 1) \rightarrow q_7$

all these states are present in middle bubble of DFA.

Reversing a Language:

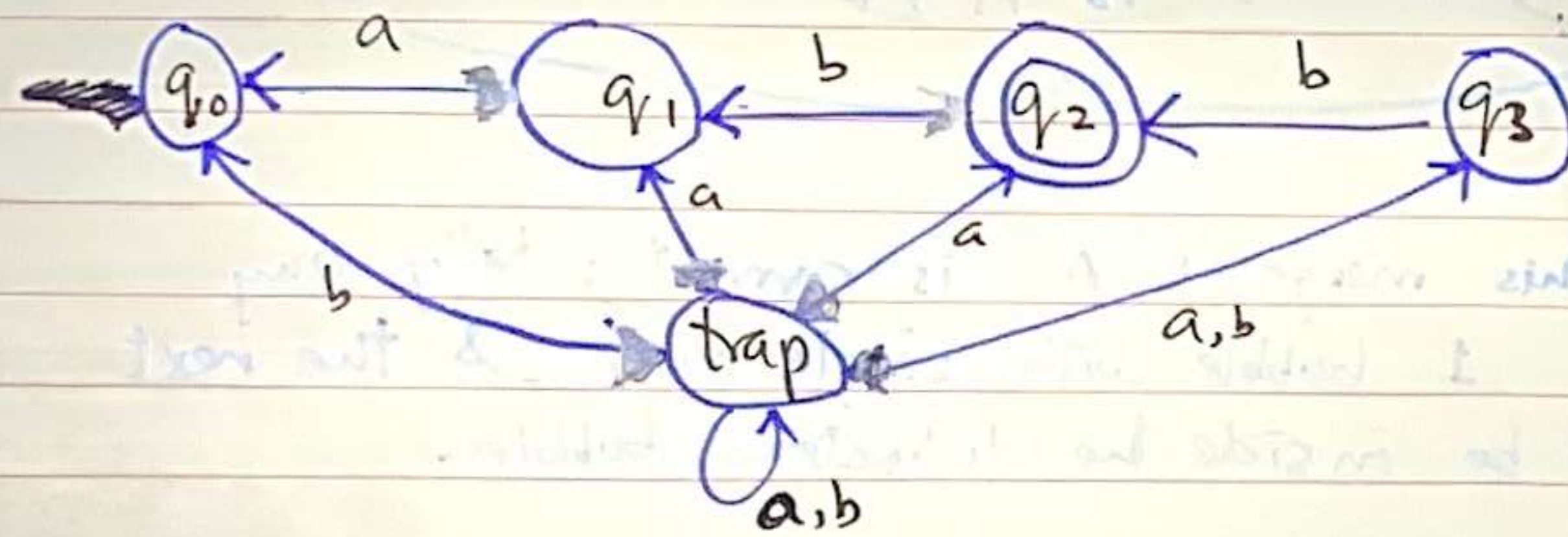
$n = \{ ab \text{ or } abb \}$

$\Rightarrow$ DFA of 'n'



$\Rightarrow$ Making DFA Diagram by reversing the arrows in above diagram

$n_{\text{Reversed}} = \{ ba \text{ or } bba \}$

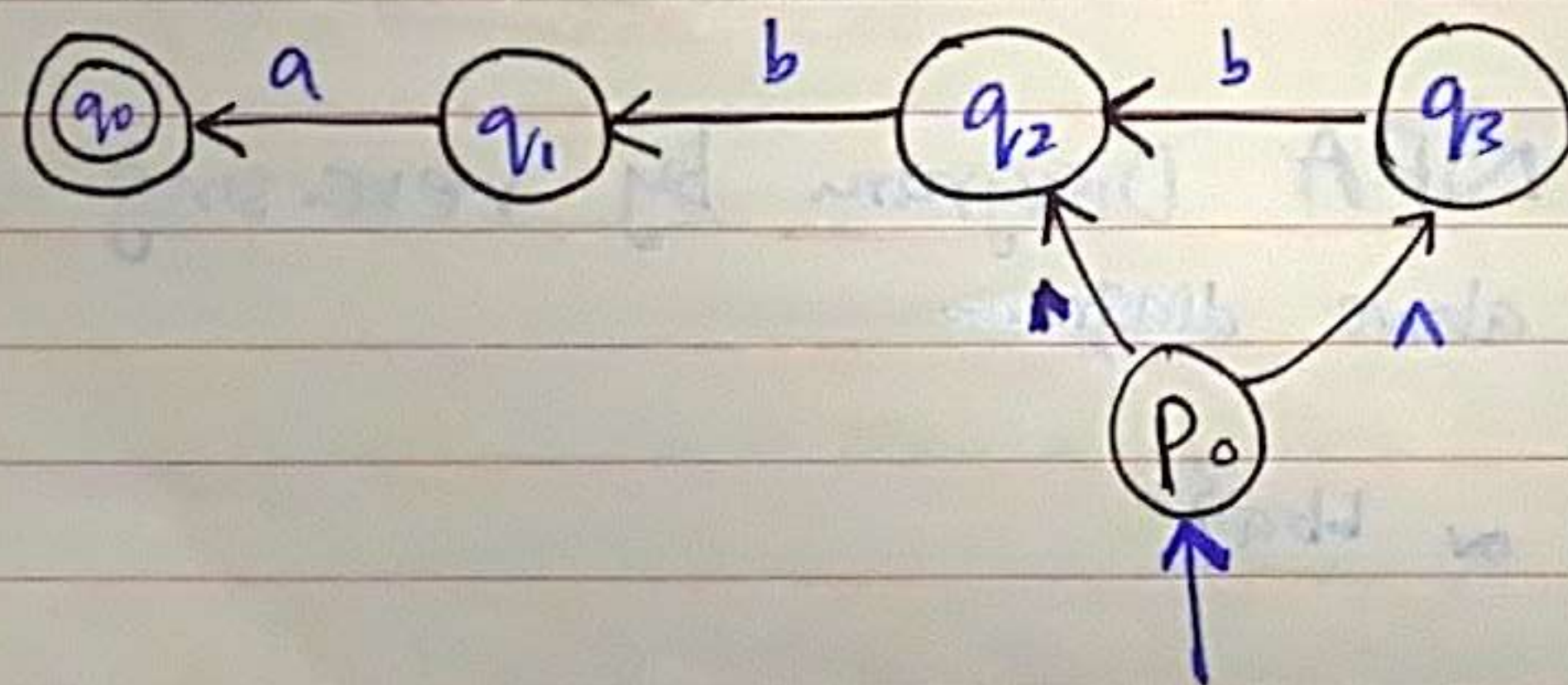


• Now we'll make initial state i.e. 'q₀' to final state and final states i.e. 'q₂ & q₃' as initial.

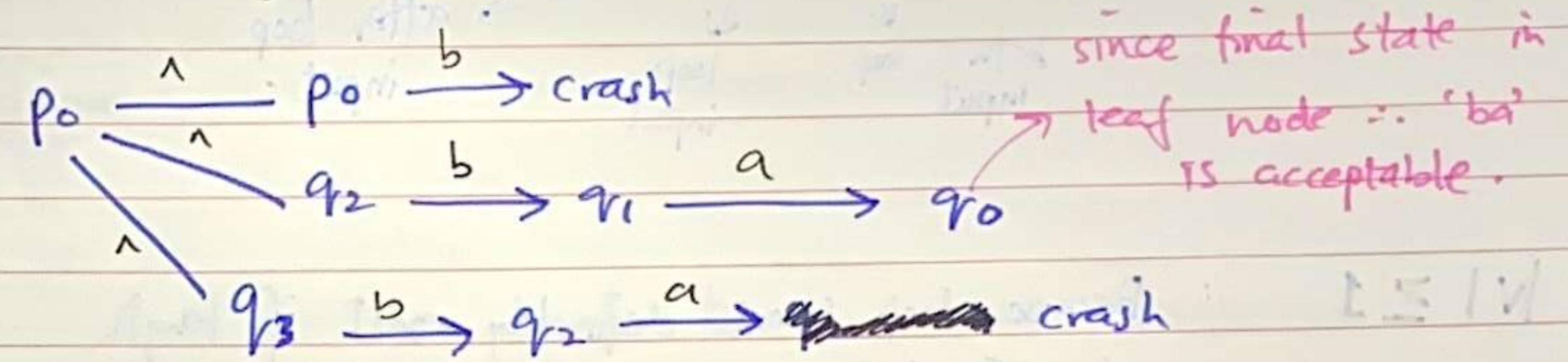
• But since there cannot be multiple initial states ∴ we'll make dummy initial state (p₀) & point it to q₂ & q₃.

• Since trap does not have any incoming arrow which means it is an inaccessible state; so we'll remove it too.

Final NFA Reversed: It became NFA from DFA.



Checking 'ba' string on previous diagram (reversed)



LECTURE - 11

☐ Pumping Lemma: To prove a lang. is Non-Regular.

~~If lang. is regular, we can make DFA with 'n' states. or [a DFA with direct or indirect loops (A → B → E → A)] if the $|x| \geq n$.~~

☒ How?

→ It is a property that a language must satisfy to be a regular language. But if we succeed to show a lang. does not satisfy Pumping Lemma; then we can say that lang. is "non-regular". (Proof By Contradiction).

⇒ YAP :

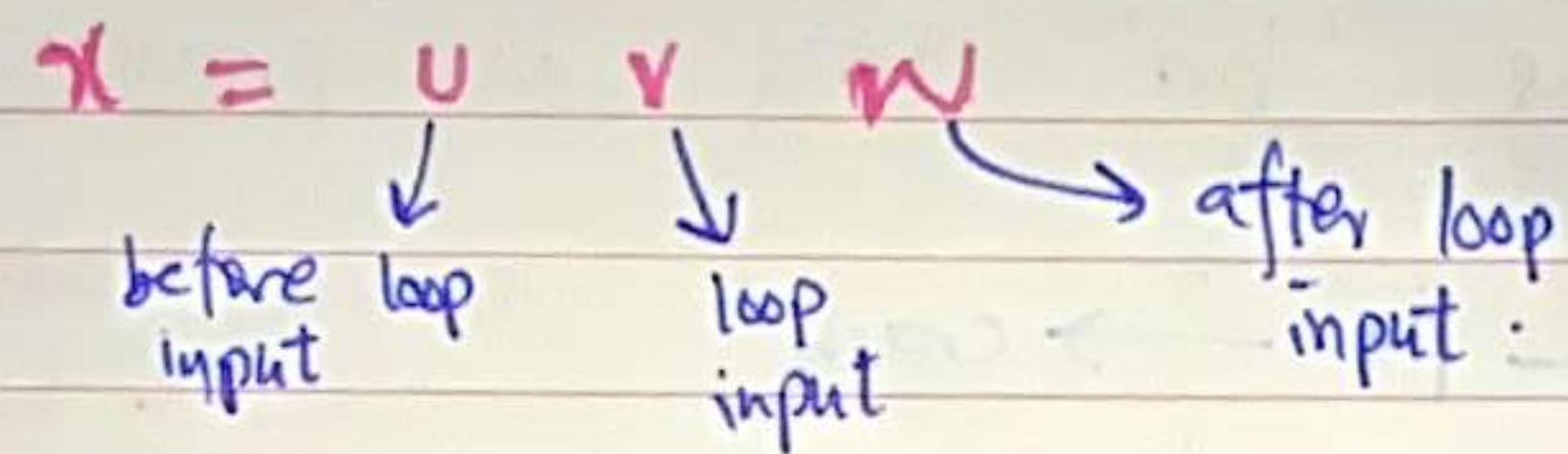
- DFA of 'n' states can be made if lang is regular with length of string x less than or equal to 'n' ($|x| \leq n$)
- But if $|x| \geq n$, its DFA can be made too obviously and that means there are direct or indirect loops (A → B → E → A)

in DFA: REGULAR

- for 'x' to be a part of language with $|x| \geq n$, we can prove the following 3 conditions to be True. OTHERWISE; that will be a NONREGULAR LANG.

* Always try to put only 1 variable in 'u' & 'v'.

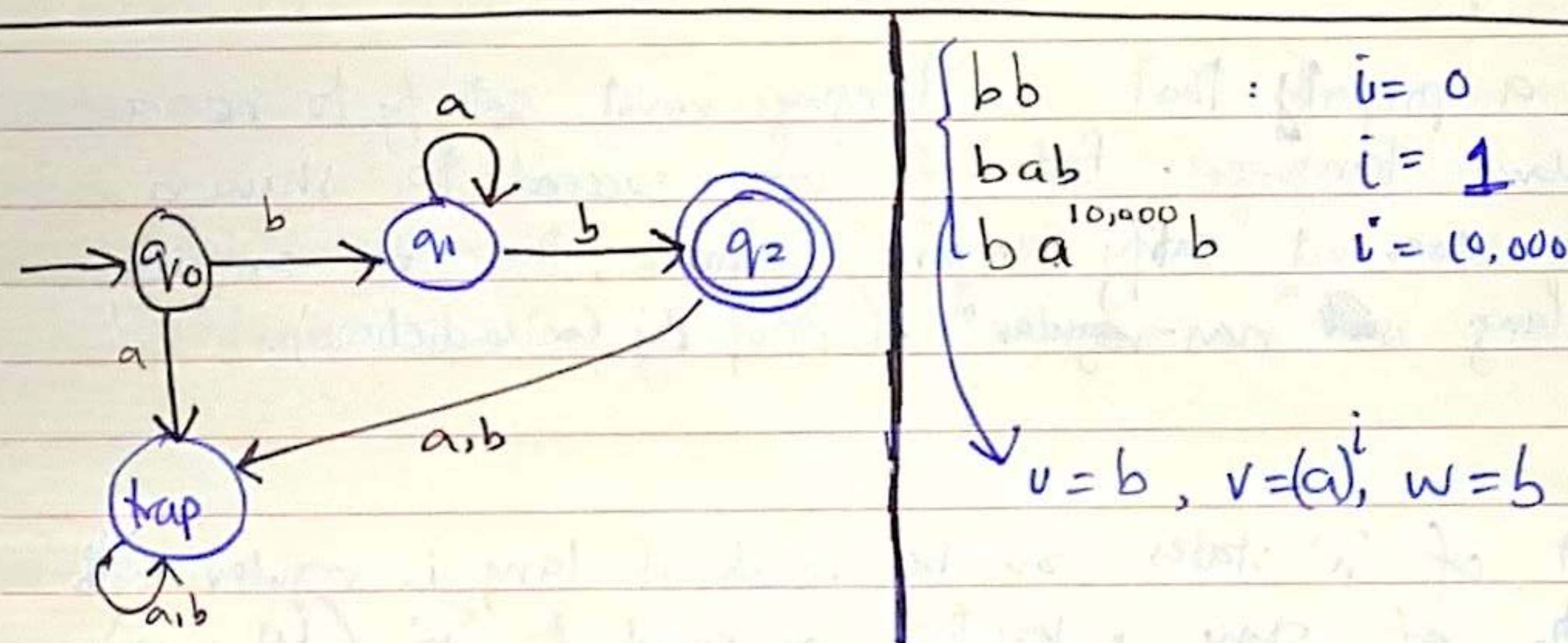
⇒ Conditions: (for lang. to be regular if $|n| \geq n$) day / date:



① $|v| \geq 1$: Because loop should definitely exist if length of string 'n' is greater than no. of states

② $|uv| \leq n$: inputs' sum before loop & including loop should be less than equal to no. of states or else it wouldn't let 'v' come in iterating part of DFA

③ For all $i \geq 0$: If it is reg. lang., the string after upper two condition with any true value of 'i' will be acceptable; otherwise non-reg. lang.



⇒ IMP POINTS:

- $|uv| = 1$, then ↓ loop exist in first/initial state (no state before initial state)
- $|uv| = n$, then ↑ loop exist either in final state or it is an indirect loop which consists of all states.

• For proving a lang. to be regular, we must make DFA since if we'll use pumping lemma, we would have to prove all cases which is humanly not possible.

day / date:

Q: Prove L_1 is Non-Regular.

$$L_1 = \{n \in \Sigma^* \text{ where } n_{ca}(x) = n_b(x)\}$$

Answer:

If L_1 is regular then for all n where $|n| \geq n$, the 3 conditions of pumping lemma should satisfy for each case.

- Therefore to prove non regularity, we'll pick a string n where $|n| \geq n$ and then will make 4 cases. In each of the cases; we'll choose specific values of 'i' for which $uv^i w \notin L$; but those cases would satisfy condition ① & ②.
- This way all possible combos of loops being direct/indirect or at start/middle/end are covered.

⇒ Choosing String: Should be greater than 'n'

eg:

$$\begin{aligned} n &= a^n b^n & n+n &= 2n & (2n \geq n) \\ n &= a^n b^n a^n b^n & & & \\ n &= a^{n/2} b a^{n/2} & n/2 + 1 + 1 + n/2 &= n+2 & (n+2 \geq n) \end{aligned}$$

$xxn = a^{10} b^{10} xx$ rejected since not in terms of 'n'

⇒ Case 1: we chose " $n = a^n b^n$ ", so now making cases.
 $|uv| = 1$ (minimum values in 'uv')
 $uv = a \therefore u = \Lambda, v = a, w = a^{n-1} b^n$

Condition ①: $|v| \geq 1$ is fulfilled as $v = a$

Condition ②: $|uv| \leq n$ is fulfilled as $|uv| = 1$

Condition ③: for all $i \geq 0$; $uv^i w \in L$

we are gonna contradict this one by choosing a specific 'i' which will show ① & ② fulfilled but ③ not fulfilled $\therefore$ non regular lang.

$$uv^i w = \Lambda \cdot (a)^i \cdot (a^{n-1} b^n)$$

$$\text{let } i=1 \rightarrow \Lambda a^1 a^{n-1} b^n = a^n b^n$$

* since $a^n b^n \in L$; we'll choose another 'i'

$$\text{let } i=0 \rightarrow \Lambda a^0 a^{n-1} b^n = a^{n-1} b^n$$

* since $a^{n-1} b^n \notin L$, $\therefore$ this case contradicts the regularity of language as 3rd condition of pumping lemma not fulfilled.

$\Rightarrow$ **Case 2**: $|uv| = n$ (max values in 'uv')

(i) $u = \Lambda, v = a^n, w = b^n$

$$uv^i w = \Lambda \cdot (a)^i \cdot b^n$$

$$\text{let } i=2 \rightarrow a^{2n} b^n$$

* since $a^{2n} b^n \notin L$; $\therefore$ contradiction.

(ii) $u = a^{n-1}, v = a, w = b^n$

$$uv^i w = a^{n-1} \cdot (a)^i \cdot b^n$$

$$\text{let } i=4 \rightarrow a^{n-1} \cdot a^4 \cdot b^n = a^{n+3} b^n$$

* since $a^{n+3} b^n \notin L$; contradiction.

(iii) $u = a^{n-k}, v = a^k, w = b^n$

$$uv^i w = a^{n-k} \cdot (a^k)^i \cdot b^n$$

$$\text{let } i=2 \rightarrow a^{n-k} \cdot a^{2k} \cdot b^n = a^{n+k} b^n$$

* since $a^{n+k} b^n \notin L$; contradiction.

$\therefore$ Since all four cases have contradiction in 3rd condition of Pumping Lemma $\therefore L_1$ is non regular.

NOTE: We could've made more cases like:
 $1 < |uv| < n$ (for example: $uv = a^{n/2}, w = a^{n/2} b^n$);
 but since Case 2 (iii) covers it so we didn't prove it.

* In paper; we need to just mention other cases like this where $|uv|$ is neither minimum nor maximum.

Need to hit & trail on choosing string since $n = a^n a^n$ can't be proved regular with any 'i'.
 Q: $L = \{u \mid u \in \Sigma^* \text{ where } u \text{ is a palindrome}\}$

* Choosing string: $u = a^n b a^n$
 $|u| = 2n+1$ ($\therefore$ greater than n)

Case 1: $|uv| = 1$

$$u = \Lambda, v = a, w = a^{n-1} b a^n$$

$$uv^i w = a^i \cdot (a^{n-1} b a^n)$$

$\therefore$ if $i=0$; $a^{n-1} b a^n \notin L$ (as not palindrome)

Case 2: $|uv| = n$

(i) $u = \Lambda, v = a^n, w = b a^n$

$$uv^i w = a^{ni} b a^n$$

$$i=0 \rightarrow b a^n \notin L$$

(ii) $u = a^{n-1}, v = a, w = b a^n$

$$uv^i w = a^{n-1} \cdot a^i \cdot b a^n$$

$$i=0 \rightarrow a^{n-1} b a^n \notin L$$

(iii) $u = a^{n-k}, v = a^k, w = b a^n$

$$uv^i w = a^{n-k} \cdot a^{ik} \cdot b a^n$$

$$i=0 \rightarrow a^{n-k} b a^n \notin L$$

$\therefore$ Since all cases contradict with " $uv^i w$ " $\therefore$ Irregular language.

LECTURE-12

Tuesday
30/9/25.

Q: Prove using Pumping Lemma; language is irregular.
 $L = \{x \mid x \in E^* \text{ \& } x = 1^a \text{ where } a \text{ is prime}\}.$

★ Since we're catering length of string (x) here
 $\therefore$ we'll play the game on length of $|uvw|$.

($1^3 = III$ → $1^7 = IIIIIII$) here are acceptable in language.

Choosing string instance

$x = 1^k$; where $k = n$ (as length of chosen instance could be equal to n as well).
 $|x| = k$ (or $|x| = n$)
 $x = uvw \therefore |uvw| = k.$

① $|uv| \leq n$

② $|v| \geq 1$

Case:

$$|uv^i w| = |uvw| + (i-1) \times |v| = |uv| + k + (i-1) \times |v|$$

★ let $i = k+1$

$$= k + (k+1-1) \times |v|$$

$$|uv^i w| = k + k|v|$$

least value of k is 2 (smallest prime)

least value of $|v| = 1$

$$\therefore |uv^i w| = 2 + 2(1) = 4$$

since $|uv^i w| = k$ where ' k ' is prime but here

we got $|uv^i w| = 4$

Since '4' isn't a prime

$\therefore$ it is non regular.

Q: Prove language is irregular.

$$L = \{x \mid x \in E^* \text{ \& } x = 1^{m^2}, \text{ where } m \geq 1\}.$$

$$m=1 \rightarrow 1^{1^2} = 1^1$$

$$m=2 \rightarrow 1^{2^2} = 1^4$$

$$m=3 \rightarrow 1^{3^2} = 1^9$$

Choosing String: $x = 1^{n^2}$

$$|x| = n^2$$

$$|uvw| = n^2$$

$uv^i w$: let $i=2$.

$$|uv^2 w| = |uvw| + |v|$$

$$|uvw| + |v| \leq n^2 + n < n^2 + 2n < n^2 + 2n + 1$$

$$\therefore n^2 < |uv^2 w| < n^2 + 2n + 1 = (n+1)^2$$

$$n^2 < |uv^2 w| < (n+1)^2$$

★ since " $uv^2 w$ " is in b/w two perfect squares
 $\therefore$ it is not part of language $\therefore$ Irregular language.

LECTURE-13

Thursday
2/10/25

Context Free Language: (CFL)

- * Every regular lang. is a CFL
- * 'Pure' CFL is for non-regular lang.

Regular lang: DFA & Regular expression

Context free lang. (pure CFL): PDA (push down automata) & CFG

Non Context free lang:

- * CFG are the set of laws that define CFL (grammar)
- * At least one tree should exist for it to be CFG.

CFG for English language

- 1 <Sentence> → <noun phrase> <verb phrase> → variables (inside <>): Not directly part of lang. only helps to define lang.
- 2 <noun phrase> → <article> <Noun>
- 3 ~~<article>~~ → the/a
- 4 <Noun> → cat/dog/man
- 5 <Verb> → is talking / is swimming → Terminals: cannot be broken down any further. They exist in lang.
- 6 <adverb> → politely.

Tree diagram for English (above):

NOTE: If a language is regular (DFA & reg expression) exist; then we'll make a reg. expression first & then convert it into CFG

- $S \rightarrow ab$
 - $S \rightarrow aa/ba$
- CFG (collection of Productions) → the lang. it referred to is CFL.
like $\{x \mid x \in \Sigma^* \text{ and } x = a^n b^n\}$

Starting variable

Set of variable (capital letters)

CFG:

$G(S, \Sigma, V, P)$

Terminals

production (set of rules)
 $S \rightarrow a^2/b^2/n$

⇒ Regular languages & CFL:

Q1) $L = \{x \mid x \in \Sigma^* \text{ and } x = a^n\}$ → one element: 'a' only.

starting variable = S

tree: $S \rightarrow a$

$S \rightarrow a$

Extra: cannot make a tree for $S \rightarrow b$ since 'b' is not part of lang.

leaf node (a) terminal

Q2) $L = \{x \mid x = a \text{ or } b\}$

$S \rightarrow a$
 $S \rightarrow b$

tree: "a" : "b" → this tree is for string 'a' or 'b'

written to show that this tree is for string 'a' or 'b'

Q3) $L = \{x \mid x \in \Sigma^* \text{ and } x = ab\}$

$S \rightarrow ab$

tree: "ab"

$S \rightarrow a \quad S \rightarrow b$

* Here the tree is read from left to right: we'll read it like 'ab' (which is part of lang).

Q4) $L = \{x \mid x \in \Sigma^* \text{ and } x = a^n\}$ → $n = \{1, a, aa, aaaa, aaaaa\}$

$S \rightarrow \Lambda$ (base case of recursion)
 $S \rightarrow aS$ (general case of recursive call to show: $S \rightarrow a/aa/aaa/aaaa$)

OR $S \rightarrow a/\Lambda$
 $S \rightarrow aS$

day / date:

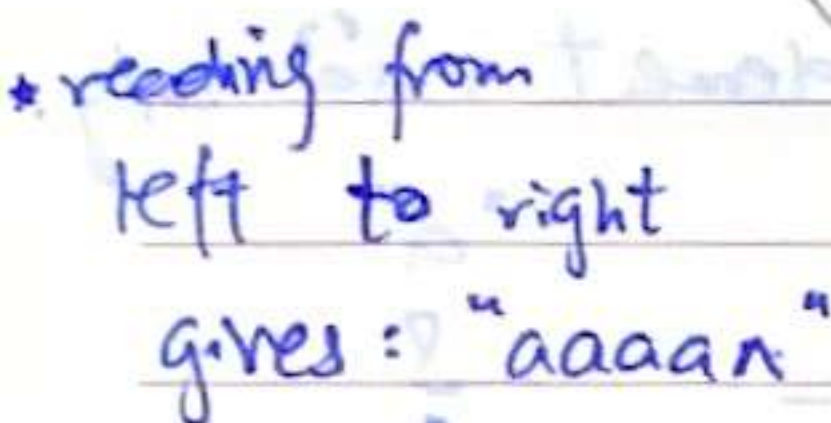
 $a^n X$ 

Diagram illustrating the expansion of the expression a^3 on the left side:

```
graph TD
    A["a³"] --> B["s"]
    A --> C["a"]
    B --> D["s"]
    B --> E["a"]
    D --> F["s"]
    D --> G["a"]
    F --> H["s"]
    F --> I["a"]
```

The diagram shows a tree structure where the root node is a^3 . It branches into s and a . The s node further branches into s and a , and this pattern continues, illustrating the expansion of the expression on the left side. An arrow points from the tree to the text "1 a a a" (expanded on left side).

It would be $\begin{matrix} s \rightarrow a \\ s \rightarrow as \end{matrix}$ NOT $\boxed{s \rightarrow as}$ since $\uparrow$

we don't have a base case i.e. $s \rightarrow a$ here), so it would be infinite call.

$$S \rightarrow \Lambda$$

$$S \rightarrow as / bs$$

$S \rightarrow \Lambda / a / b$
 $S \rightarrow S$

tree =

tree:

abaa

$$abaan = abaa$$

day / date:

① Reg. expression : $(a+b)^*ab$

previously: $S \rightarrow as/b5$

$$S \rightarrow ab : :$$

$$S \rightarrow AB$$

$$A \rightarrow \Lambda / aA / bA$$
$$B \rightarrow ab$$

tree :

"aab"

A B

* NOTE (Method) → We need to break our string (i.e. aacb) ourself to map the divided parts of it into our tree.

 ~~$a \wedge b$~~ $a \wedge ab = aab$

OR using only one variable.

② $s \rightarrow ab$ (base case reach when ends with 'ab')
 $s \rightarrow as / bs$

Q7) $L = \{ n \mid n \text{ starts with } ab \}$

$$S \rightarrow BA$$
$$A \rightarrow \cdot A \mid aA \mid bA$$
$$B \rightarrow ab$$

of

$$S \rightarrow ab$$
$$S \rightarrow Sa/Sb$$

's' is in start

Swapping rows here does not

change expression at all.

1st line 3n4 Base

condition always